

3 - Java

Introduzione

Java è un linguaggio di programmazione orientato agli oggetti e indipendente dalla piattaforma di esecuzione, garantita dal fatto che il codice non dipende dalla macchina fisica o dal sistema operativo specifico, ma viene compilato in un formato intermedio chiamato **bytecode**, il quale può essere poi eseguito su qualunque Virtual Machine compatibile. A differenza del paradigma procedurale, che descrive i sistemi come un insieme di processi tramite flow-chart e utilizza procedure e funzioni tipiche di linguaggi come C, Basic o Pascal, il mondo a oggetti descrive i sistemi come un insieme di "cose" modellate attraverso gerarchie e dipendenze tra classi. Questo moderno approccio utilizza costrutti basati su dichiarazioni di classi e metodi.

Il paradigma Object-Oriented

Riprendendo la definizione del paradigma orientato agli oggetti, ogni singola entità del mondo reale è considerata un oggetto caratterizzato da uno stato e da un funzionamento specifico.

☰ Esempio di paradigma OO nel mondo reale

Ad esempio, lo stato di una bicicletta è definito da valori come la marcia corrente e la velocità di marcia, mentre il suo funzionamento comprende azioni concrete come il cambio della marcia o la frenata

Nel contesto del software, gli oggetti reali vengono modellati memorizzando il loro stato all'interno di **fields** (ovvero variabili o attributi) ed esponendo il loro funzionamento all'esterno attraverso **methods** (funzioni o metodi), che hanno il compito cruciale di modificare lo stato dell'oggetto agendo in modo mirato sui suoi attributi.

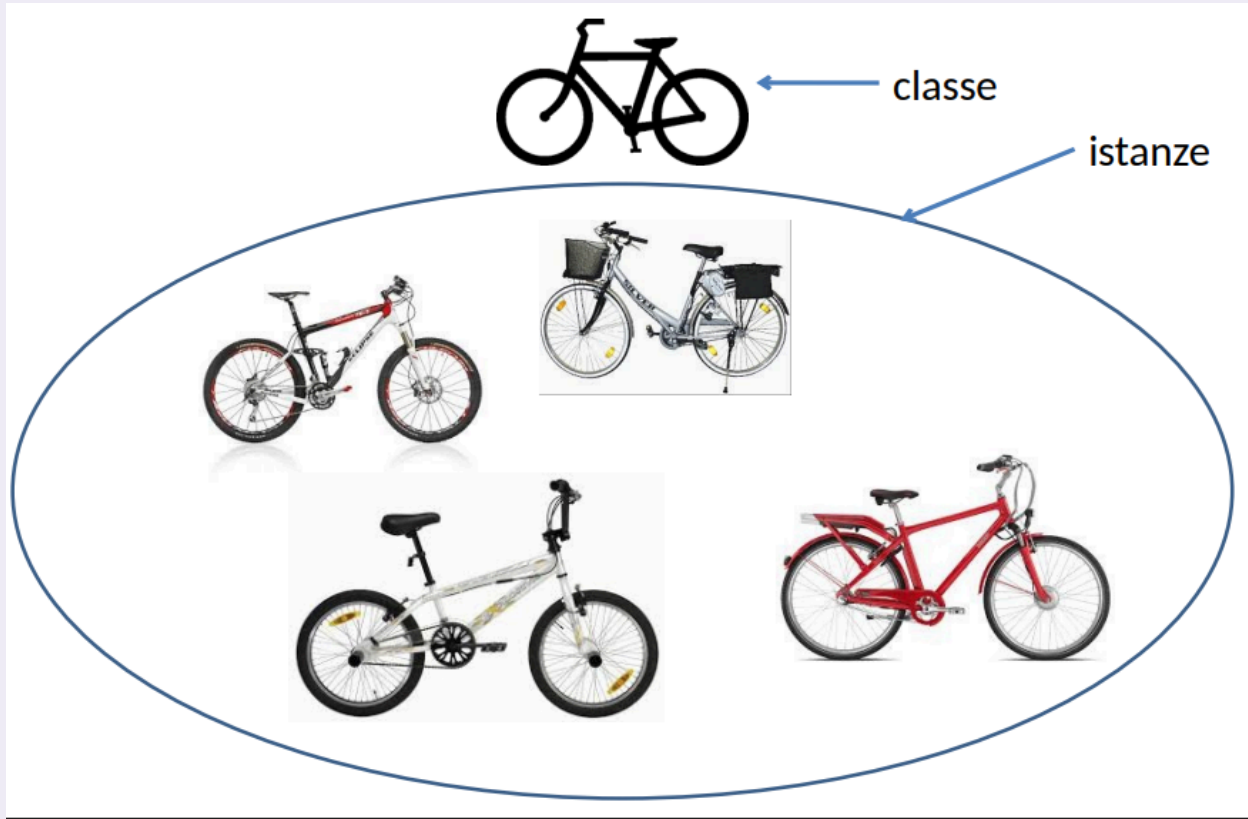
L'adozione di questo paradigma porta numerosi e importanti vantaggi:

- Favorisce la modularità, permettendo di gestire il codice di ogni oggetto in modo separato e indipendente;
- Garantisce l'information-hiding, nascondendo i dettagli implementativi e permettendo l'interazione solo tramite metodi prestabiliti;
- Facilita il riutilizzo del codice, consentendo di estendere o sostituire facilmente oggetti scritti da altri sviluppatori.

Classi, Ereditarietà e Package

Poiché nel mondo reale moltissimi oggetti condividono delle caratteristiche intrinseche, nella programmazione essi vengono raggruppati in **classi** che ne definiscono il funzionamento e le peculiarità comuni, rendendo quindi i singoli oggetti istanze fisiche e operative di tali classi.

☰ Esempio di classe

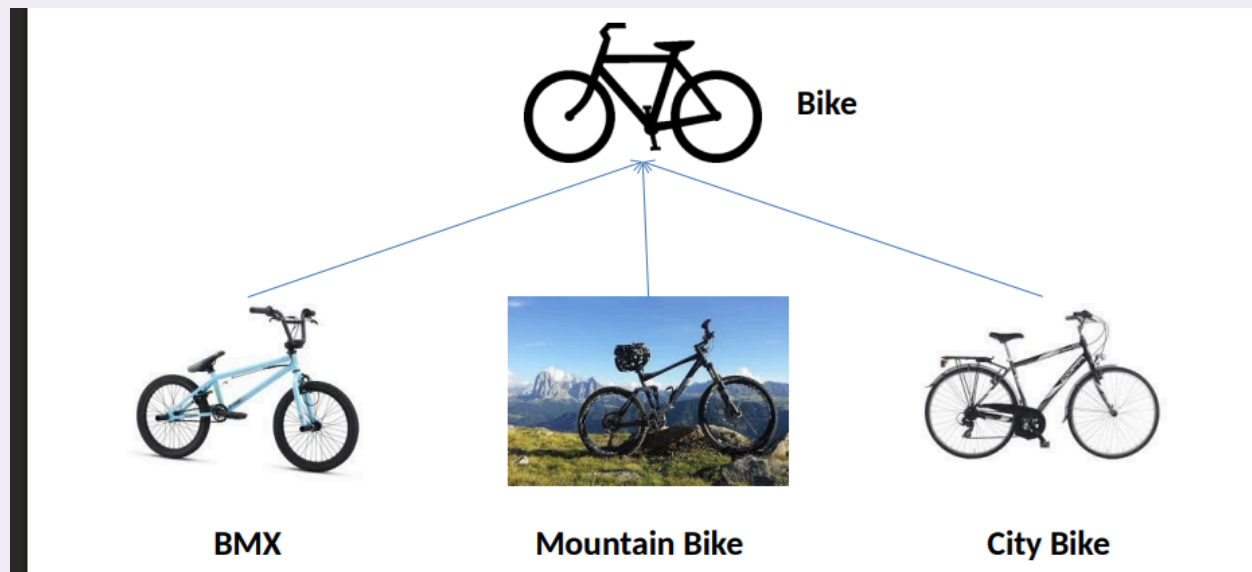


L'ereditarietà è un meccanismo estremamente potente che permette di rappresentare le gerarchie tra classi, consentendo a una determinata classe di ereditare in blocco gli attributi e i metodi di un'altra classe genitore.

☰ Esempio di ereditarietà

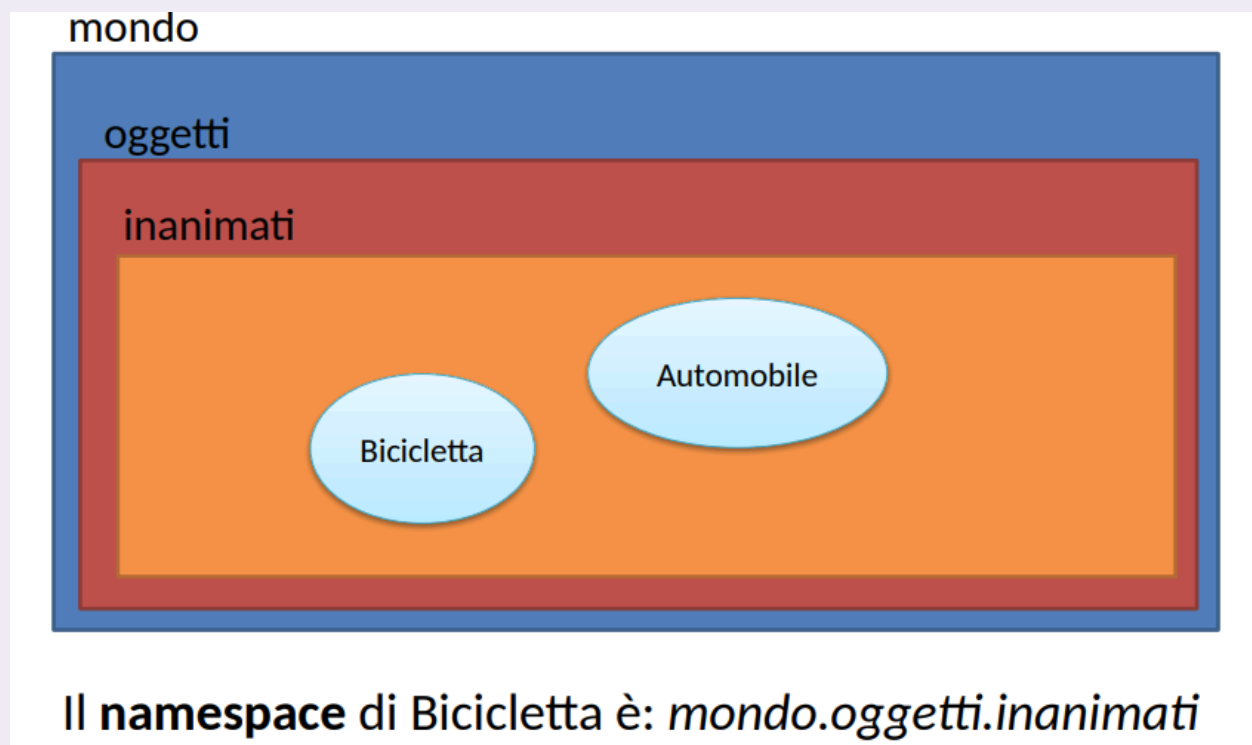
Riprendendo l'esempio di prima, classi più specifiche come le BMX, le Mountain Bike e le City Bike ereditano tutte le caratteristiche di base di una Bicicletta generica,

aggiungendo poi i propri attributi e metodi specializzati.



Per gestire l'organizzazione di progetti complessi, Java permette di strutturare le classi all'interno di specifiche cartelle separate chiamate **package**, le quali supportano la creazione di ulteriori sottocartelle per definire gerarchie annidate. Il percorso completo e sequenziale dei package necessari per individuare univocamente una classe prende il nome formale di **namespace**

☰ Rappresentazione grafica di package e namespace sulle biciclette



Codice

Commenti

Per commentare dentro un codice Java, utilizziamo questi simboli:

```
/*
Questo è un commento multi riga, infatti posso anche andare a capo senza
farmi problemi.
Ecco qui!
*/

//Questo è un commento per una singola riga
```

Il compilatore Java ignorerà qualunque cosa posto all'interno degli slash per le righe con la prima sintassi, nella seconda soltanto la riga corrente

Definizione di classi, il main e println()

Per definire una classe in Java usiamo la sintassi **class <nomeclasse>**

☰ Esempio di definizione classe

```
class HelloWorldApp{
    public static void main(String[] args){
        System.out.println("Ciao mondo!")
    }
}
```

Il main è l'entrypoint di ogni applicazione dove possiamo richiamare altre classi e metodi, in Java deve essere sempre presente all'interno di un applicazione. Come argomento ha sempre `String[] args`, che si riferisce agli argomenti che possiamo passare al main dalla riga di comando quando invochiamo un applicazione Java.

La riga `System.out.println("Ciao mondo!")` è composta da 3 elementi fondamentali:

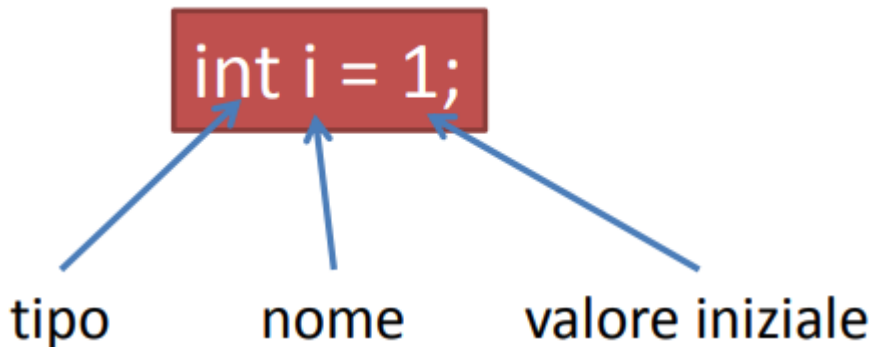
- `System`, che è una delle librerie **core** di Java
- `out` è un suo attributo (in questo caso un oggetto `PrintStream`)
- `println` è un metodo della classe `PrintStream` che permette di scrivere su un dispositivo di output

Le variabili

Le variabili in Java ricoprono il ruolo fondamentale di **descrivere lo stato di un oggetto**, ed esse possiedono sempre:

- Un tipo;
- Un nome identificativo;
- Un valore coerente con il tipo specificato in fase di dichiarazione.

Vanno sempre dichiarate, utilizzando la seguente sintassi:



Nel linguaggio esistono diverse categorie di variabili:

- Le variabili delle istanze (non statiche), in cui ogni singolo oggetto conserva il suo stato in modo indipendente;
- Le variabili delle classi (statiche), che risultano condivise e identiche per tutte le istanze di una specifica classe;
- Le variabili locali, che vengono utilizzate e risultano accessibili esclusivamente all'interno del metodo in cui sono dichiarate;
- I parametri, che rappresentano i valori passati al momento della chiamata di un metodo

Per quanto riguarda i nomi identificativi, il linguaggio impone che siano case-sensitive, che inizino rigorosamente con una lettera o con il simbolo \$ e che possano contenere numeri al loro interno, avendo preferibilmente delle nomenclature autoesplicative.

Le convenzioni comunemente accettate prevedono che

- Le costanti e le variabili statiche debbano essere scritte interamente a lettere maiuscole,
- Per i nomi composti da più parole si adotta lo stile in cui la prima lettera di ogni nuova parola successiva alla prima viene scritta in maiuscolo.

Tipi di dato primitivi

Il linguaggio Java definisce a livello nativo una serie di tipi di dato primitivi per ottimizzare la memorizzazione dei valori.

Tabella dei valori primitivi

Tipo	Descrizione
byte	8-bit, intero [-128, 127], valore di default 0
short	16-bit, intero [-32.768, 32.767], valore di default 0
int	32-bit, intero [-2^{31} , $2^{31}-1$], valore di default 0
long	64-bit, intero [-2^{63} , $2^{63}-1$], valore di default 0
float	32-bit IEEE 754 floating point, valore di default 0
double	64-bit IEEE 754 floating point, valore di default 0
boolean	true/false, valore di default <i>false</i>
char	16-bit Unicode character, \u0000-\uFFFF, valore di default \u0000

Quasi tutti i tipi primitivi possiedono uno zero numerico, un carattere nullo o il valore false come impostazione di default, ad eccezione delle variabili locali che risultano prive di tale assegnazione automatica e necessitano pertanto di un'inizializzazione manuale.

Ogni tipo primitivo è inoltre affiancato dalla sua rispettiva classe "wrapper" (la sua rappresentazione ad oggetti come Integer, Long o Boolean).

Tipo String

Il tipo String non si tratta di un tipo primitivo ma di un oggetto vero e proprio il cui valore di default risulta essere null. Le stringhe sono istanze immutabili e il linguaggio provvede a creare automaticamente un nuovo oggetto String ogni volta che incontra una sequenza di caratteri racchiusa tra doppi apici.

I letterali

Il termine letterale (**literal**) viene utilizzato per indicare i **valori espliciti che assegniamo direttamente nel codice alle variabili di tipo primitivo o alle stringhe**.

☰ Esempi di literals

```
boolean r = true;
char c = 'Z';
byte b = 100;
short s = 10000;
int i = 100000;
```

Letterali interi

Per quanto riguarda i letterali interi, il compilatore li interpreta sempre come tipo `int` di default, a meno che non venga specificato esplicitamente l'uso del formato long tramite l'aggiunta di una lettera L finale; essi supportano inoltre la scrittura in formati alternativi, potendo essere espressi anche in base esadecimale o binaria.

☰ Esempio di letterali interi

```
int n=10;
long m=1000L;
int h=0xa1;
int b=0b001101;
```

Letterali in virgola mobile (o floating point)

I letterali in virgola mobile seguono una logica simile: sono considerati di tipo `double` per impostazione predefinita, a meno che non si utilizzi la lettera `f` per forzarne il tipo a `float`, e supportano l'impiego della notazione scientifica.

☰ Esempio di letterali float

```
float f=3.14f;
double d1=134.54
double d2=1.3454e2 //Notazione scientifica
```

Letterali caratteri e Stringhe (character e String)

I character possono contenere qualunque carattere Unicode a 16-bit racchiuso tra singoli apici, mentre le stringhe impiegano i doppi apici.

☰ Esempio di letterali caratteri e stringhe

```
Per char vale questo -> 'c'
Per String vale questo -> "testo di esempio"
```

All'interno dei testi è inoltre possibile avvalersi di speciali sequenze di escape per rappresentare agevolmente caratteri di controllo come il line feed, la tabulazione oppure per visualizzare simboli specifici come il doppio apice, il singolo apice e il backslash

📄 Lista di sequenze escape

```
\n, \r, \f, \b: line feed, carriage return, form feed, backspace
\t: tab
\\', \', \\: doppio apice, singolo apice, backslash
```

Array

L'array è un **oggetto** che contiene un numero finito di oggetti (o tipi primitivi) dello stesso tipo.

Ha una lunghezza fissa definita al momento della creazione, ogni suo elemento viene chiamato **elemento** ed è possibile accederne con l'indice

☰ Esempio di vari tipi di array

```
Dichiarazione di un array: float[] v;
Inizializzazione: float[] v=new float[100];
Assegnazione: v[3]=1.3;
Dichiarazione e inizializzazione contemporanea: int[] a = {10, 34,
21};
Array multidimensionali: double[][] m;
```

Quando si dichiara un array inizializzato, questo avrà dimensioni uguali a quanti elementi son stati inizializzati

Copia di un array

La classe `System` mette a disposizione il metodo `arraycopy` per copiare degli array

Metodo per la copia di un array

```
arraycopy(Object src, int srcPos, Object dest, int destPos, int length)
```

È composto da:

- **src**: array sorgente;
- **srcPos**: posizione di inizio in src;
- **dest**: array di destinazione;
- **destPos**: posizione di inizio in dest;
- **length**: numero di elementi da copiare;

Esempio di copia di array

```
int[] a = {10, 30, 20, 15, 45};  
int[] b = new int[3];  
System.arraycopy(a, 2, b, 0, 3);
```

```
b[0] = 20  
b[1] = 15  
b[2] = 45
```

Manipolazione di array

La classe `Arrays` mette a disposizione un serie di metodi statici per manipolare gli array, la documentazione è presente [qui](#), ma riportiamo comunque alcuni metodi come:

- **copyOfRange**: è la copia di array;
- **fill**: riempimento di array;

- **equals**: confronto tra due array;
- **search**: ricerca di un elemento;
- **sort**: ordinamento crescente;

Operatori

Gli operatori eseguono delle operazioni da 1 a 3 argomenti e restituiscono un valore. Gli operatori con stessa priorità son eseguiti da sinistra verso destra, **tranne per le operazioni di assegnamento**

Tabella operatori

Operatori	Precedenza
postfix	expr++, expr--
unary	++expr, --expr, +expr, -expr, ~, !
multiplicative	*, /, %
additive	+, -
shift	<<, >>, >>>
relational	<, >, <=, >=, instanceof

Operatori	Precedenza
equality	==, !=
bitwise AND	&
bitwise XOR	^
bitwise OR	
logical AND	&&
logical OR	
ternary	?, :
assignment	=, +=, -=, *=, /=, %=, &=, ^=, =, <<=, >>=, >>>=

Alcune considerazioni da fare sono:

- **unary**: permette di negare un numero binario;
- **shift**: shifta i bit in un numero
- **relational**: mettono in relazione due
- **equality**: i doppi uguale son particolari, con i tipi primitivi funzionano nella classica maniera, mentre nel confronto di oggetti funzionano solo se questi ultimi hanno lo stesso OID (Object ID)

Operatori aritmetici

Tabella degli operatori aritmetici

+	somma	int s = 3 + 4;
-	differenza	int d = 3 - 2;
*	moltiplicazione	int m = 3 * 2;
/	divisione	int d = 10 / 2;
%	modulo	int r = 11 % 5;

È possibile combinare assegnazione ed operatori aritmetici, per esempio:

```
x += 1; è equivalente a x = x + 1;
x *= 2; è equivalente a x = x * 2;
```

L'operatore `+` può essere utilizzato per concatenare le stringhe:

```
String a = "Hello ";
String b = " world!";
String msg = a + b; ("Hello world!")
```

Operatori unari

Tabella degli operatori unari

+	indica i numeri positivi, può essere omesso	int a=+1;
-	indica i numeri negativi	int b=-a;
++	incremento	a++;
--	decremento	b--;
!	complemento logico, inverte il valore di un boolean	boolean b = true; boolean a = !b;

Operatori uguaglianza e relazionali

Operatore	Nome
==	uguale a
!=	diverso da
>	maggiore di
>=	maggiore o uguale a
<	minore di
<=	minore o uguale a

Questi operatori restituiscono un boolean come risultato del confronto tra due espressioni

Operatori condizionali

Operatore	Nome
&&	AND-logico
	OR-logico

In Java le espressioni sono valutate solo se necessario, per esempio se ci sono OR valuterà soltanto una e, se questa è vera, le altre verranno scartate a prescindere (in altri linguaggi questo non succede)

Operatore 'instanceof'

instanceof è un particolare operatore per stabile se un oggetto è istanza di una particolare classe, come risultato restituisce un booleano.

Esempio di instanceof


```
String s = "sono una stringa";
boolean t = s instanceof String;
boolean f = s instanceof Double
```

Bitwise e bitshift

Gli operatori **bitwise** e di **bit shift** permettono di **manipolare direttamente i singoli bit a livello macchina**.

L'operatore tilde (~) calcola il complemento della rappresentazione binaria andando a trasformare tutti gli zeri in uno e viceversa.

Per quanto riguarda lo scorrimento dei bit (shift) troviamo:

- L'operatore << sposta i bit verso sinistra
- L'operatore >> li sposta verso destra
- L'operatore >>> si occupa di spostare i bit a destra ma senza conservare il segno originale

Java mette a disposizione specifici operatori binari sui bit, che sono l'AND logico, lo XOR e l'OR che troviamo all'inizio del capitolo .

Espressioni

Un'espressione in Java è un costrutto fondamentale composto tipicamente da **variabili**, **operatori** e **chiamate a metodi**

La caratteristica essenziale di un'espressione è che, una volta valutata, restituisce sempre un singolo risultato.

Il tipo di questo risultato non è universale, ma dipende strettamente dai tipi delle variabili coinvolte, dagli operatori utilizzati e, naturalmente, dai valori restituiti dagli eventuali metodi chiamati.

Per gestire priorità complesse, le parentesi possono essere utilizzate liberamente per definire e forzare un preciso ordine di valutazione degli operatori.

Blocchi

I blocchi sono gruppi di istruzioni racchiusi tra parentesi graffe

☰ Esempio di blocco di codice

```
if (condition) {
    // Inizio primo blocco
    System.out.println("La condizione è vera.");
}
```

```

}
// Fine primo blocco
else {
// Inizio secondo blocco
System.out.println("La condizione è falsa.");
}
// Fine secondo blocco

```

Statement

Uno **statement** rappresenta una **singola istruzione esecutiva** e la sua sintassi richiede tassativamente che termini sempre con il carattere punto e virgola (;).

All'interno del linguaggio si possono individuare quattro tipologie principali di statement:

1. Le istruzioni di assegnazione di valori;
2. Gli operatori matematici particolari come ++ o --;
3. Le istruzioni che effettuano una chiamata ad un metodo;
4. Le istruzioni dedicate alla creazione di un nuovo oggetto in memoria;

Controllo del flusso

if-then e if-then-else

Le istruzioni if-then e if-then-else sono essenziali per eseguire dei blocchi di codice solo ed esclusivamente se si verifica una determinata condizione.

☰ Esempio di istruzione if-then-else

```

if (condition) {
//blocco if
}
else {
//blocco else
}

```

L'if-then avviene soltanto quando una condizione è vera, altrimenti nel caso sia falsa questa va nell'else

if-else annidati

Quando la logica del programma lo richiede, è assolutamente possibile annidare più istruzioni if-else per creare percorsi decisionali multipli

```
if (condizione1) {
    //blocco condizione 1
}
else if (condizione2){
    //blocco condizione 2
}
else if (condizione3){
    //blocco condizione 3
}
```

Switch

Per gestire scenari con molteplici opzioni si utilizza l'istruzione **switch**, che definisce diversi percorsi di esecuzione in base allo specifico valore che assume una singola variabile

≡ Esempio di switch

```
switch (<variabile>) {
    case <valore1>: ...; break; // istruzione 1
    case <valore2>: ...; break; // istruzione 2
    default: ...; break; // istruzione default
}
```

Tale variabile viene confrontata con vari blocchi "case", prevedendo anche un caso "default" finale nel caso nessuna condizione sia rispettata.

Lo switch può essere applicato soltanto a tipi di dati ben precisi, ovvero ai tipi primitivi byte, short, int, char, alla classe String e ai tipi enumerativi

While

Il ciclo while permette di eseguire un blocco di istruzioni finché una specifica condizione posta in testa al ciclo si mantiene vera

≡ Esempio di condizione while

```
while (condizione) {
    //blocco istruzioni while
}
```

}

Do while

Il ciclo do-while è concettualmente simile al while, ma con una profonda differenza, ossia il controllo della condizione viene effettuato solamente alla fine del blocco di codice, facendolo sempre eseguire **almeno** una volta.

≡ Esempio do-while

```
do {
    //blocco istruzioni while
} while (condizione);
```

for

L'istruzione **for** permette di iterare un blocco di istruzioni su uno specifico intervallo di valori

≡ Esempio for

```
for (initialization; termination; increment) {
    statement(s) //blocco istruzioni
}
```

L'intestazione di questo ciclo racchiude tre parametri chiave separati da punto e virgola: la fase di inizializzazione, la condizione di terminazione e l'incremento (o decremento)

for, Array, Collection (for each)

Il for è spesso utilizzato per iterare sugli oggetti di un Array o di una Collection, in questo caso esiste una forma compatta (**for-each**)

≡ Esempio for-each

```
public static void main(String[] args){
    int[] numbers ={1,2,3,4,5,6,7,8,9,10};
    for (int item : numbers) {
        System.out.println("Count is: " + item);
    }
}
```

```
    }
}
```

A ogni passaggio del ciclo, la variabile temporanea `item` assume automaticamente il valore dell'elemento successivo, dal primo all'ultimo, fermandosi da sola quando l'array è terminato

Il `for-each` **non garantisce nessun ordinamento**, serve soltanto a visitare quello presente all'interno del contenitore

break

Lo statement `break` interrompe istantaneamente e definitivamente un ciclo

≡ Esempio di statement break

```
for (i = 0; i < arrayOfInts.length; i++) { //si cerca
    if (arrayOfInts[i] == searchfor) {
        foundIt = true;
        break;
    }
}
```

Nel caso di cicli annidati, è importante sapere che il `break` va a interrompere solamente il ciclo più innestato rispetto alla posizione dell'istruzione stessa,

continue

Lo statement `continue` salta la corrente iterazione per passare a quella successiva

≡ Esempio di statement continue, cercando in una stringa solo le 'p'

```
for (int i = 0; i < max; i++) {
    // interested only in p's
    if (searchMe.charAt(i) != 'p')
        continue;
    // process p's
    numPs++;
}
```

Spesso viene attivato da un costrutto condizionale, come in questo caso

return

L'istruzione `return` viene adoperata per terminare in modo definitivo l'esecuzione di un metodo corrente.

A seconda del metodo, può essere usata in modo autonomo e senza restituire alcun valore, oppure restituendo un valore specifico tramite una variabile

Classi

Dichiarazione di una classe

La dichiarazione di una classe in Java serve a definire la struttura fondamentale che comprende lo stato della classe attraverso gli attributi, le modalità di creazione e inizializzazione tramite i costruttori, e le funzionalità offerte mediante i metodi

Definizione di una classe in Java

```
class MyClass {  
    // attributi  
    // costruttori  
    // metodi  
}
```

A livello sintattico, la dichiarazione di una classe inizia specificando la sua visibilità, seguita dalla parola chiave `class` e dal nome della classe stessa

Definizione di una classe completa

```
<visibilità> class <nome> extends <superclass>  
implements <interface1>, <interface2> ... {  
    //body  
}
```

Durante questa fase è anche possibile indicare da quale superclasse essa erediti, utilizzando la parola chiave **`extends`**, e definire se implementa una o più interfacce tramite la parola chiave **`implements`** (le vedremo successivamente).

Il livello di visibilità, che può essere `public`, `private` o `protected`, determina come e da chi la classe può essere vista e utilizzata all'interno del progetto

Variabili all'interno

All'interno di una classe possiamo trovare diverse tipologie di variabili:

- I **field**, o **attributi**, sono variabili che definiscono lo stato dell'oggetto e risultano visibili in tutta la classe
- Le **variabili locali**, sono confinate e visibili esclusivamente all'interno del metodo in cui vengono dichiarate e utilizzate
- I **parametri** sono variabili speciali che vengono passate a un metodo nel momento in cui questo viene richiamato

Dichiarazione degli attributi

Quando si dichiara un attributo, è necessario specificarne la visibilità, il tipo di dato e il nome; se non viene assegnato esplicitamente un valore iniziale, la variabile assumerà il valore di default previsto per il suo tipo

≡ Esempio di variabili all'interno di una classe

```
public class Bicycle {  
    private int gear = 1;  
    private int speed;  
}
```

Metodi

Dichiarazione

I metodi rappresentano le funzioni di una classe e la loro dichiarazione richiede l'indicazione della visibilità, del tipo di dato restituito, del nome e di eventuali parametri racchiusi tra parentesi

≡ Esempio di dichiarazione di metodi

```
public int sum(int a, int b) { //tipo, nome e parametri  
    return a+b;  
}
```

Nome

Per convenzione, i nomi dei metodi dovrebbero essere scritti in minuscolo e iniziare preferibilmente con un verbo, poiché indicano un'azione; se il nome è composto da più parole, si utilizza la notazione camelCase, inserendo la lettera maiuscola per identificare l'inizio di ogni parola successiva alla prima

Overloading di metodi

Java supporta l'overloading dei metodi, una funzionalità che permette a una classe di avere più metodi con lo stesso nome, a condizione fondamentale che abbiano una lista di parametri differente per numero o tipo.

È importante notare che i metodi sovraccaricati con lo stesso nome devono comunque restituire lo stesso tipo di valore.

☰ Esempio di overloading

```
public int sum(int a, int b) {  
    return a+b;  
}  
  
public int sum(int a, int b, int c) {  
    return a+b+c;  
}
```

Parametri

Per quanto riguarda i parametri, ogni metodo può riceverne da zero a molti, e questi possono essere di qualsiasi tipo, spaziando dai tipi primitivi fino agli oggetti di altre classi come array o stringhe

I parametri all'interno della firma del metodo devono avere nomi differenti tra loro e non è consentito dichiarare variabili locali all'interno del metodo che abbiano lo stesso nome di un parametro. È invece possibile che un parametro abbia lo stesso nome di un attributo della classe, tuttavia in questo scenario l'attributo viene "nascosto" e reso non direttamente visibile al metodo, a meno che non si utilizzi in modo esplicito la parola chiave **this** per disambiguare il riferimento.

Costruttori

I costruttori sono metodi speciali progettati appositamente per inizializzare gli oggetti di una classe al momento della loro creazione.

☰ Esempio di un costruttore


```
// Metodo senza valori
public Bicycle(int startSpeed, int startGear) {
    gear = startGear;
    speed = startSpeed;
}
// Metodo con valori
public Bicycle() {
    gear = 1;
    speed = 0;
}
```

Essi si distinguono per avere lo stesso identico nome della classe a cui appartengono e per il fatto di non restituire alcun valore, nemmeno void.

Esattamente come accade per i metodi, una classe può dichiarare più di un costruttore sfruttando l'overloading, purché non esistano due costruttori con lo stesso numero e tipo di parametri. Anche ai costruttori possono essere applicati i modificatori di visibilità **private**, **public** o **protected**.

Istruzione 'this'

L'istruzione **this** è uno strumento fondamentale che permette di fare riferimento in modo esplicito agli attributi e ai costruttori della classe corrente in cui ci si trova, utile per risolvere ambiguità di nomi o per richiamare un costruttore dall'interno di un altro costruttore

≡ Esempio di istruzione 'this'

```
public class Bicycle {
    private int gear;
    private int speed;
    public Bicycle(int startSpeed, int startGear) {
        this.gear = startGear;
        this.speed = startSpeed;
    }

    public Bicycle() {
        this(3, 0);
    }
}
```

Oggetti

Gli oggetti veri e propri prendono vita in memoria utilizzando l'istruzione **new**, la quale si occupa di invocare il costruttore appropriato della classe a cui l'oggetto deve appartenere. Una volta che l'oggetto è stato istanziato, è possibile accedere ai suoi attributi e invocare i suoi metodi sfruttando la sintassi basata sul punto.

≡ Esempio di istanza e utilizzo di un oggetto

```
Bicycle myBike = new Bicycle(0, 3); //Istanza di un nuovo oggetto

myBike.gear; //richiamo dell'attributo
myBike.getSpeed(); //Richiamo del metodo
```

Numero arbitrario di argomenti

Un'ulteriore caratteristica avanzata permette di definire in un metodo un numero arbitrario di argomenti dello stesso tipo. In questo caso, all'interno del metodo, il parametro viene trattato a tutti gli effetti come un array, rappresentato dai 3 puntini

≡ Esempio di numero arbitrario di argomenti

```
public void print(String... s) { // s è visto come un array
    for (String item:s) {
        System.out.println(item);
    }
}

print("Pippo", "Topolino", "Pluto") // Posso invocare print passando
    tanti oggetti di tipo String
```

Metodi static

Il modificatore **static** viene impiegato per definire attributi e metodi a livello di classe anziché a livello di singola istanza. Questo significa che una variabile dichiarata come static è condivisa e mantiene lo stesso valore per tutte le istanze create da quella specifica classe. Di conseguenza, i metodi contrassegnati come static possono essere chiamati direttamente utilizzando il nome della classe, senza la necessità preventiva di creare un'istanza o un oggetto.

☰ Esempio con dichiarazione ed uso metodi static

```
public class Person {
    static int numbOfPersons=0; // Qui il numero di persone aumenterà
    // senza essere resettato alla prossima esecuzione
    private String name;
    private String surname;
    public Person(String name, String surname) {
        this.name=name;
        this.surname=surname;
        ++numbOfPersons;
    }
}

Person p1=new Person(''Pippo'', ''Rossi'');
Person p2=new Person(''Topolino'', ''Bianchi'');
System.out.println(Person.numbOfPersons); // Accedo al valore della
// variabile static della classe Person
```

Costanti

Per definire delle **costanti**, ovvero attributi il cui valore una volta assegnato non può più essere modificato, si utilizza il modificatore **final**. Un uso comune in Java per le costanti globali prevede la combinazione dei modificatori **public**, **static** e **final**, rendendo il valore accessibile ovunque senza dover istanziare la classe

```
private final int A = 3; //Visibile solo nella classe
public final int X = 4 // Visibile ad altre classi
public static final double PI=3.14 // Visibile ad altre classi in modo
// static (senza dover creare un'istanza della classe)
```

Classi innestate

Il linguaggio offre la possibilità di definire una classe direttamente all'interno del corpo di un'altra classe, creando le cosiddette **classi innestate**.

Queste si dividono principalmente in due categorie:

- **Classi innestate statiche:** Una classe innestata statica è indipendente e non ha accesso alle risorse e ai membri della classe esterna

- **Classi interne:** al contrario, una classe interna gode di un accesso privilegiato a tutte le risorse della classe esterna che la contiene, comprese quelle dichiarate con visibilità private

L'utilizzo delle classi innestate è consigliato principalmente in tre scenari:

- Quando la classe interna risulta utile esclusivamente alle logiche della classe esterna;
- Quando si desidera incapsulare una classe B dentro una classe A per permetterle di accedere alle risorse private di A mantenendole sicure;
- In generale per raggruppare logicamente il codice rendendolo più pulito e leggibile.

☰ Esempio di classi innestate

```
class OuterClass {
    ...
    static class StaticNestedClass { // Non ha accesso alle risorse di
OuterClass
        ...
    }
    class InnerClass { // Ha accesso alle risorse di OuterClass anche
se dichiarate private
        ...
    }
}
```

Tipi enumerativi

I tipi enumerativi permettono di definire dei tipi che possono assumere solo un set predefinito di costanti

☰ Esempio di tipo enumerativo

```
public enum Day {
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY
}
```

Quando si crea un tipo enum, Java crea automaticamente una classe di tipo enum che mette a disposizione dei metodi e permette di definire anche dei costruttori e nuovi metodi.

Interfacce

Le interfacce in Java permettono di definire il funzionamento di una classe indicando esplicitamente quali metodi la classe deve possedere per potervi aderire. Esse contengono unicamente la descrizione delle firme dei metodi senza fornirne alcuna implementazione, ma possono comunque includere delle costanti.

Dichiarazione di interfacce

```
public interface <nome> extends <interface1>, <interface2>,
<interface3> {
    // dichiarazioni di costanti
    // firme dei metodi
    void methodA(int i, double x);
    int methodB(String s);
}
```

Grazie alle interfacce è possibile definire gruppi di classi che condividono le stesse funzionalità, lasciando però a ciascuna di esse la libertà di implementarle in maniera differente

L'implementazione pratica da parte di una classe avviene tramite la parola chiave `implements`, la quale obbliga la classe stessa a fornire il codice per tutti i metodi definiti all'interno dell'interfaccia.

Nelle interfacce va rispettato le stesse regole di nome delle classi

Ereditarietà

In Java, le classi possono ereditare da altre classi, il che implica che la sottoclasse acquisisce automaticamente tutti gli attributi e i metodi che possiedono una visibilità `public` oppure `protected` all'interno della superclasse

Esempio di ereditarietà in una calcolatrice

Prendiamo una calcolatrice basica, come questa:

```
public class Calculator {  
    private double memory;  
    public Calculator() {  
    }  
    public Calculator(double memory) {  
        this.memory = memory;  
    }  
    public double getMemory() {  
        return memory;  
    }  
    public void setMemory(double memory) {  
        this.memory = memory;  
    }  
    public void resetMemory() {  
        this.setMemory(0);  
    }  
}
```

```
public double sum(double a, double b) {  
    return a + b;  
}  
public double diff(double a, double b) {  
    return a - b;  
}  
public double mul(double a, double b) {  
    return a * b;  
}  
public double div(double a, double b) {  
    return a/b;  
}  
}
```

Decidiamo di estenderla con una calcolatrice scientifica

```
public class ScientificCalculator extends Calculator {
    public double tan(double x) {
        return Math.tan(x);
    }
    public double sin(double x) {
        return Math.sin(x);
    }
}
```

ScientificCalculator estende Calculator. Quindi sarà di tipo Calculator ed erediterà tutti gli attributi e i metodi pubblici di Calculator

```
ScientificCalculator sc=new ScientificCalculator();
System.out.println(sc.sin(4));
System.out.println(sc.sum(3, 4));
```

ScientificCalculator eredita il metodo pubblico *sum* della classe padre Calculator

Un aspetto fondamentale del linguaggio è che tutte le classi scritte in Java ereditano, in modo diretto o indiretto, dalla classe madre universale chiamata `Object`

Overriding e poliformismo

Una classe ha la capacità di ridefinire un metodo che ha precedentemente ereditato dalla sua superclasse (o classe padre).

Quando questo accade, il metodo originale della superclasse viene nascosto e, al suo posto, viene invocato esclusivamente quello ridefinito all'interno della sottoclasse; questo specifico fenomeno strutturale prende il nome di **overriding**

Il polimorfismo è strettamente legato ai concetti di ereditarietà e overriding, in quanto rappresenta la **capacità di ogni sottoclasse di poter ridefinire specifici comportamenti della propria superclasse**.

Tutte le sottoclassi possono riscrivere determinati comportamenti per adattarli alle proprie esigenze, pur mantenendo altre caratteristiche operative in comune con la classe **padre**

≡ Esempio di overriding, poliformismo e ereditarietà

```
public class ClassA {
    public void printMe() {
        System.out.println("Io sono A");
    }
    public void sayHello() {
        System.out.println("Hello!");
    }
}

public class ClassB extends ClassA {
    public void printMe() { //override del metodo in A, ereditato
        System.out.println("Io sono B");
    }
}
```

Se qui andassi a richiamare il metodo specificando un oggetto di tipo B e metodo printMe, otterrei come output "Io sono B", poichè ho fatto overriding del metodo precedente

Accesso alla superclasse da una sottoclasse

Per interagire esplicitamente con gli elementi della superclasse, Java mette a disposizione dello sviluppatore la parola chiave `super`, questa quando viene utilizzata permette:

- Di invocare direttamente i costruttori della classe padre, utilizzando `super()` senza parametri o `super(lista parametri)` con argomenti specifici, \
- Di richiamare i metodi originali della superclasse tramite la sintassi `super.methodSuper(...)`

≡ Esempio di accesso alla superclasse da una sottoclasse

```
public class ClassA {
    public void printMe() {
        System.out.println("Io sono A");
    }
    public void sayHello() {
        System.out.println("Hello!");
    }
}
```

```

}

public class ClassB extends ClassA {
    public void printMe() {
        super.printMe() //qui invochiamo il metodo di A direttamente
        System.out.println("Io sono B");
    }
}

```

Classi astratte

Le classi astratte si distinguono per la presenza di metodi astratti, ovvero funzioni di cui viene fornita solamente la firma senza alcuna implementazione, sebbene la classe possa contenere dei metodi implementati

La caratteristica restrittiva principale di una classe astratta, dichiarata tramite la parola chiave `abstract`, è che non è fisicamente possibile istanziarne degli oggetti diretti, infatti esse sono concepite esclusivamente per essere ereditate; in tal caso, la sottoclasse ha l'obbligo di fornire un'implementazione concreta per tutti i metodi astratti che ha ereditato

≡ Esempio di dichiarazione classe astratta

```

public abstract class GraphicObject {
    // declare fields
    // declare nonabstract methods
    abstract void draw();
}

```

Classi astratte vs interfacce

Esistono differenze importanti tra classi astratte e interfacce:

1. Nelle classi astratte è possibile dichiarare attributi che non siano obbligatoriamente `static` e `final`, e si possono definire metodi completi della loro implementazione. All'interno delle interfacce tutti gli attributi devono essere rigorosamente `static` e `final`, e tutti i metodi assumono di default una visibilità `public`
2. In Java è consentito estendere una sola classe alla volta (anche se astratta), ma si possono implementare contemporaneamente numerose interfacce
3. Le classi astratte sono utili quando si desidera condividere del codice (sotto forma di metodi) tra un insieme di classi che sono tra di loro strettamente correlate, quando ci si

aspetta che le classi figlie abbiano molti metodi o attributi in comune, o quando si necessita di attributi non statici e non finali per permettere ai metodi di modificare liberamente lo stato degli oggetti. Le interfacce sono la scelta preferibile se le classi che le devono implementare non sono strettamente correlate, quando si vuole semplicemente specificare il comportamento di una particolare struttura dati senza entrare nei dettagli implementativi, o quando si ha il bisogno specifico di ricorrere e simulare un'ereditarietà multipli.

Classi astratte e Interfacce combinate

In Java, una classe astratta può implementare una o più interfacce. Il grande vantaggio in questo caso è che la classe astratta non è obbligata a implementare subito tutti i metodi richiesti, si può decidere di scriverne solo alcuni e lasciarne altri non implementati, le sottoclassi concrete (cioè le classi "normali" che andranno a estenderla) avranno poi l'obbligo di fornire il codice per tutti i metodi rimasti vuoti.

Numeri

Java mette a disposizione delle classi che rappresentano i tipi primitivi numerici che sono `Byte`, `Short`, `Long`, `Integer`, `Float`, `Double`. Ognuno di questi eredita dalla classe madre `Numbers`.

Il compilatore converte automaticamente tra tipi primitivi e classi usando una tecnica chiamata **boxing/unboxing**:

- Il **boxing** (noto anche come **wrapper**) è la trasformazione che consiste nel posizionare un tipo primitivo all'interno di un oggetto in modo che il valore possa essere utilizzato come riferimento.
- **Unboxing** è la trasformazione inversa dell'estrazione del valore primitivo dal suo oggetto nel wrapper

Questa classe mette a disposizione come sottoclassi:

- `BigInteger/BigDecimal`: sottoclassi adibite a interi e decimali con alta precisione
- `AtomicInteger/AtomicDecimal`: sottoclassi adibite a interi e decimali per applicazioni concorrenti, una volta che un'operazione viene effettuata su un tipo di questo genere non è interrompibile (da questo la sua atomicità)

Di solito si usa questa classe per alcuni motivi, tra cui:

- Un argomento deve essere un oggetto e non un tipo primitivo
- Quando bisogna definire delle costanti statiche nelle classi (come `MAX_VALUE` o `MIN_VALUE`)
- Per delle conversioni tra tipo, come
 - Da `String` (stringa contenente dei numeri) a tipo primitivo

- Da un tipo numerico ad un altro

La classe number implementa diversi metodi tra cui:

Metodi per la conversione in un tipo primitivo

- `byte byteValue()`
- `short shortValue()`
- `int intValue()`
- `long longValue()`
- `float floatValue()`
- `double doubleValue()`

Metodi per il confronto di un numero con un argomento

- `int compareTo(Byte anotherByte)`
- `int compareTo(Double anotherDouble)`
- `int compareTo(Float anotherFloat)`
- `int compareTo(Integer anotherInteger)`
- `int compareTo(Long anotherLong)`
- `int compareTo(Short anotherShort)`

`boolean equals(Object obj)`

Questo metodo serve per confrontare due oggetti booleani per determinare se siano di stesso valore. Restituisce "true" se sono uguali, altrimenti "false"

Metodi per la conversione

- `static Integer decode(String s)`: converte una stringa in `Integer`
- `static int parseInt(String s)`: converte una stringa in `int`
- `static int parseInt(String s, int radix)`: converte una stringa in `int`, (`radix`=sistema di numerazione 10, 2, 8, 16)
- `String toString()`: restituisce la stringa che rappresenta questo numero
- `static String toString(int i)`: restituisce la stringa che rappresenta il numero `i`
- `static Integer valueOf(int i)`: restituisce l'`Integer` che rappresenta il valore primitivo `i`
- `static Integer valueOf(String s)`: simile a `parseInt` ma restituisce un `Integer`
- `static Integer valueOf(String s, int radix)`: simile a `parseInt`

Stampa dei numeri

In Java, poiché ogni numero può essere convertito in una stringa, è possibile stamparli direttamente sullo standard output utilizzando i metodi `System.out.print` e `System.out.println`. `System.out` è un'istanza della classe `PrintStream`, il che permette di utilizzare indifferentemente anche i metodi equivalenti `printf` e `format`

☰ Esempio di format

```
System.out.format("The value of the float variable is %f, while the
value of the integer variable is %d, and the string is %s", floatVar,
intVar, stringVar)
```

Il metodo `format` accetta una stringa di formattazione seguita da una serie di argomenti, specificando in modo preciso come questi ultimi debbano essere visualizzati

📄 Flag converter e flag

Al metodo `format` possono essere passate queste flag:

Converter	Flag	Explanation
d		A decimal integer
f		A float
n		A new line character appropriate to the platform running the application. You should always use <code>%n</code> , rather than <code>\n</code>
tB		A date & time conversion—locale-specific full name of month
td, te		A date & time conversion—2-digit day of month. td has leading zeroes as needed, te does not
ty, tY		A date & time conversion—ty = 2-digit year, tY = 4-digit year
tl		A date & time conversion—hour in 12-hour clock.
tM		A date & time conversion—minutes in 2 digits, with leading zeroes as necessary.
tp		A date & time conversion—locale-specific am/pm (lower case).
tm		A date & time conversion—months in 2 digits, with leading zeroes as necessary.
tD		A date & time conversion—date as <code>%tm%td%ty</code>
	08	Eight characters in width, with leading zeroes as necessary.
	+	Includes sign, whether positive or negative.
	,	Includes locale-specific grouping characters.
	-	Left-justified..
	.3	Three places after decimal point.
	10.3	Ten characters in width, right justified, with three places after decimal point.

Math

La classe `Math` mette a disposizione strumenti avanzati per eseguire svariate operazioni matematiche. Al suo interno si trovano:

- Costanti fondamentali come `Math.E` e `Math.PI`
- Metodi per operazioni di base, tra cui:
 - Calcolo del valore assoluto tramite `abs`
 - Arrotondamento tramite `round`
 - Determinazione del valore minimo o massimo tra due numeri con `min` e `max`
 - `double ceil(double d)` per ottenere l'intero più piccolo che sia maggiore o uguale al parametro
 - `double floor(double d)` per calcolare l'intero più grande che sia minore o uguale
 - `double rint(double d)` per trovare l'intero più vicino a `d`

Sono presenti metodi per calcoli esponenziali e logaritmici come:

- `double exp(double d)` per calcolare e^d
- `double log(double d)` per il logaritmo naturale $\ln(d)$
- `double pow(double base, double exponent)` per eseguire l'elevamento a potenza di una base per un esponente
- `double sqrt(double d)` per calcolare la radice quadrata

Per le funzioni trigonometriche di base e inverse troviamo altri metodi come:

- `sin`, `cos`, `tan`, `asin`, `acos`, `atan`
 - `double atan2(double y, double x)` che permette di convertire coordinate rettangolari in coordinate polari, restituendo l'angolo theta calcolato
 - `double toDegrees(double d)`, e `toRadians(double d)` converte l'argomento in gradi o radianti
- Come argomento per ogni metodo viene passato un `double` che rappresenta l'angolo espresso in radianti

Per generare numeri casuali nell'intervallo 0 - 1 si usa `Math.random()`, il cui risultato può essere scalato moltiplicandolo per un intero. Nel caso si voglia generare una serie più articolata di numeri casuali è consigliato l'utilizzo della classe dedicata `java.util.Random`

Stringhe

La classe `String` rappresenta sequenze testuali ed è caratterizzata dalla sua immutabilità, ossia il valore dell'istanza non può in alcun modo essere alterato una volta portato a compimento il suo processo di creazione.

Ogni letterale scritto tra virgolette nel codice Java è a tutti gli effetti una vera e propria istanza predefinita della classe `String`

Una stringa può essere pensata e gestita logicamente come un array di caratteri; infatti, per accedere agli elementi si usa il metodo `charAt()`, che restituisce il carattere posizionato all'indice specificato

☰ Esempio di charAt()

```
char[] helloArray = { 'h', 'e', 'l', 'l', 'o', '.' };
String helloString = new String(helloArray);
System.out.println(helloString);
```

Il metodo `charAt(int i)` di `String` restituisce il carattere alla *i*-esima posizione

Metodi della classe `String`

La classe presenta diversi metodi:

- `length()` restituisce la lunghezza della stringa in caratteri
- `contains(CharSequence s)` verifica la presenza di una particolare sequenza testuale con risultato booleano
- `indexOf(String s)` restituisce l'indice dal quale inizia la sottostringa `s`, -1 se non esiste la sottostringa

- `indexOf(String s, int i)` restituisce l'indice dal quale inizia la sottostringa `s` a partire dall'*i*-esimo carattere, -1 se non esiste la sottostringa
- `replace(CharSequence s1, CharSequence s2)` consente di scambiare sequenze di caratteri esatti (`s1` è quella da cercare, `s2` quella con cui sostituirla)
- `matches(String regex)` restituisce true se la stringa corrisponde all'espressione regolare `regex`
- `startsWith` ed `endsWith` verificano in modo rapido le sequenze collocate agli estremi di una stringa, vengono usati in congiunzione ad altri metodi
- `equals(Object o)` è l'operatore di uguaglianza fatto per le stringhe, si deve assolutamente evitare l'uso dell'operatore di uguaglianza classico tra oggetti (`==`)
- `int compareTo(String str)`, `int compareToIgnoreCase(String str)` permettono un confronto funzionale all'ordinamento, il secondo si usa quando non è necessario il case-sensitive
- `substring(int b, int e)`, `substring(int b)` permette di estrarre e generare una porzione della stringa a partire da un indice iniziale fino alla fine, o limitandosi a un indice finale escluso
- `trim` rimuove gli spazi bianchi inutili presenti all'inizio e alla fine
- `toLowerCase()`, `toUpperCase()` convertono globalmente il formato delle lettere rispettivamente in minuscolo e maiuscolo

≡ Esempio di stringa palindroma

```
public class StringDemo {
    public static void main(String[] args) {
        String palindrome = "Dot saw I was Tod";
        int len = palindrome.length();
        char[] tempCharArray = new char[len];
        char[] charArray = new char[len];
        // La stringa originale viene messa in un array di char
        for (int i = 0; i < len; i++) {
            tempCharArray[i] = palindrome.charAt(i);
        }
        // L'array di char viene girato
        for (int j = 0; j < len; j++) {
            charArray[j] = tempCharArray[len - 1 - j];
        }
        //Viene messa in un nuovo oggetto e stampata
        String reversePalindrome = new String(charArray);
        System.out.println(reversePalindrome);
    }
}
```



```
}
}
```

Conversione da numeri a stringhe

La conversione da un formato stringa verso effettivi valori numerici è operabile utilizzando i metodi di parsing specifici implementati per ogni tipo primitivo:

```
int i = Integer.parseInt("42");
float f = Float.parseFloat("3.14");
double d = Double.parseDouble("4.32144");
Float fo = Float.valueOf("3.14");
```

Se l'esigenza è quella di trasformare un numero nativo in una stringa, si farà ricorso ai metodi di conversione:

```
int i=3; double d=3.4;
String s3 = Integer.toString(i);
String s4 = Double.toString(d);

int i=3;
String s1 = String.valueOf(i);
```

Classe StringBuilder

Ogni qual volta sia necessario costruire o manipolare stringhe di cui si vuole modificare costantemente il contenuto, è mandatorio ricorrere alla classe `StringBuilder`

La classe lavora con una lunghezza variabile che consente l'aggiunta o la modifica continua di nuovi caratteri

Metodi e costruttori StringBuilder

Metodi di StringBuilder

`length()` restituisce la lunghezza della stringa presente nell'oggetto `StringBuilder`
`capacity()` restituisce la capacità attuale di questo oggetto `StringBuilder` (`>=length()`)

Costruttori di StringBuilder

Costruttore	Descrizione
<code>StringBuilder()</code>	Crea uno <code>StringBuilder</code> vuoto
<code>StringBuilder(CharSequence cs)</code>	Crea uno <code>StringBuilder</code> a partire da una sequenza di caratteri
<code>StringBuilder(int initCapacity)</code>	Crea uno <code>StringBuilder</code> con una capacità iniziale <i>initCapacity</i>
<code>StringBuilder(String s)</code>	Crea uno <code>StringBuilder</code> a partire da una stringa <i>s</i>

- `StringBuilder append(...)` aggiunge alla fine dello `StringBuilder` l'argomento (viene convertito in `String` anche i tipi primitivi)
- `StringBuilder delete(int start, int end)`, `StringBuilder deleteCharAt(int index)` cancella una porzione o un carattere della stringa
- `StringBuilder insert(int offset, ...)`: inserisce l'argomento a partire dalla posizione `offset`
- `StringBuilder replace(int start, int end, String s)`, `void setCharAt(int index, char c)` sostituisce una porzione o un singolo carattere della stringa
- `StringBuilder reverse()` inverte l'ordine dei caratteri della stringa
- `String toString()`: restituisce una stringa che contiene la sequenza dei caratteri in `StringBuilder`

≡ Esempio con `Stringbuilder`, stringa palindroma

```
public class StringBuilderDemo {
    public static void main(String[] args) {
        String palindrome = "Dot saw I was Tod";
        StringBuilder sb = new StringBuilder(palindrome);
        sb.reverse(); // reverse it
        System.out.println(sb);
    }
}
```

Espressioni regolari

Un'**espressione regolare** rappresenta a livello logico una parola capace di denotare un linguaggio regolare, utile per verificare se una data stringa sia o meno conforme alle regole di quel linguaggio.

📘 Diverse espressioni regolari nel linguaggio Java

Espressioni regolari (caratteri)

x	The character <i>x</i>
\\	The backslash character
\0n	The character with octal value <i>0n</i> ($0 \leq n \leq 7$)
\0nn	The character with octal value <i>0nn</i> ($0 \leq n \leq 7$)
\0mnn	The character with octal value <i>0mnn</i> ($0 \leq m \leq 3, 0 \leq n \leq 7$)
\xhh	The character with hexadecimal value <i>0xhh</i>
\uhhhh	The character with hexadecimal value <i>0xhhhh</i>
\t	The tab character (' <code>\u0009</code> ')
\n	The newline (line feed) character (' <code>\u000A</code> ')
\r	The carriage-return character (' <code>\u000D</code> ')
\f	The form-feed character (' <code>\u000C</code> ')
\a	The alert (bell) character (' <code>\u0007</code> ')
\e	The escape character (' <code>\u001B</code> ')
\cx	The control character corresponding to <i>x</i>

28

Espressioni regolari (chars classes)

- [abc]** a, b, or c (simple class)
- [^abc]** Any character except a, b, or c (negation)
- [a-zA-Z]** a through z or A through Z, inclusive (range)
- [a-d[m-p]]** a through d, or m through p: [a-dm-p] (union)
- [a-z&&[def]]** d, e, or f (intersection)
- [a-z&&[^bc]]** a through z, except for b and c: [a-dz] (subtraction)
- [a-z&&[^m-p]]** a through z, and not m through p: [a-lq-z](subtraction)

29

Espressioni regolari (default classes)

- . Any character (may or may not match line terminators)

\d A digit: [0-9]

\D A non-digit: [^0-9]

\s A whitespace character: [\t\n\x0B\f\r]

\S A non-whitespace character: [^\s]

\w A word character: [a-zA-Z_0-9]

\W A non-word character: [^\w]

30

Espressioni regolari (boundary)

^ The beginning of a line

\$ The end of a line

\b A word boundary

\B A non-word boundary

\A The beginning of the input

\G The end of the previous match

\Z The end of the input but for the final terminator, if any

\z The end of the input

31

Espressioni regolari (greedy operator)

$X?$ X , once or not at all

X^* X , zero or more times

X^+ X , one or more times

$X\{n\}$ X , exactly n times

$X\{n,\}$ X , at least n times

$X\{n,m\}$ X , at least n but not more than m times

32

Espressioni regolari (quotation)

$\backslash$ Nothing, but quotes the following character

$\backslash Q$ Nothing, but quotes all characters until $\backslash E$

$\backslash E$ Nothing, but ends quoting started by $\backslash Q$

33

Espressioni regolari (logical operators)

XY X followed by Y

$X|Y$ Either X or Y

(X) X , as a capturing group

34

Dalla classe `String`, due metodi sono utilizzabili con le varie espressioni regolari:

- `split(String regex)` restituisce un array di `String` dividendo dove c'è il match con `regex`
- `replaceAll(String regex, String r)` sostituisce tutte le sequenze che corrispondono all'espressione regolare `regex` con `r`

Classe Pattern

Per utilizzi che richiedono procedure di validazione più strutturate o intensive, si ricorre esplicitamente alla classe `Pattern`, la quale costituisce una rappresentazione compilata in memoria di un'espressione regolare specificata tramite stringa

Partendo da un pattern già compilato, è possibile generare un oggetto derivato di tipo `Matcher`, il cui compito primario è quello di eseguire attivamente il matching, cercando in tutti i modi di far combaciare la sequenza di caratteri passata in input con le direttive fornite dall'espressione regolare pre-compilata

≡ Esempio di invocazione

```
Pattern p = Pattern.compile("a*b");
Matcher m = p.matcher("aaaaab");
```

```
boolean b = m.matches();
```

Classe Matcher

La classe `Matcher` esegue le operazioni di matching su una sequenza di caratteri in funzione dell'espressione regolare compilata

Un matcher viene creato da un pattern invocando il metodo `matcher` del pattern. Una volta creato, un matcher può essere utilizzato per eseguire tre diversi tipi di operazioni di match:

- `match` viene usato per tentare l'abbinamento esatto dell'intera sequenza in input
- `lookingAt` cerca un abbinamento parziale avendo però l'obbligo di partire rigorosamente dall'inizio
- `find` serve a scorrere e analizzare progressivamente l'input alla ricerca della prima sottosequenza utile che corrisponda positivamente

Ognuno di questi metodi restituisce un valore booleano che indica l'esito positivo o negativo.

Un matcher trova corrispondenze in un sottoinsieme del suo input chiamato regione. Per impostazione predefinita, la regione contiene tutto l'input del matcher. Una regione può essere modificata tramite il metodo `region` e costantemente monitorata interrogando i valori forniti da `regionStart` e `regionEnd`

Lo stato esplicito di un matcher include gli indici di inizio e fine dell'ultimo `match` (`start` ed `end`), includendo anche gli indici di inizio e fine della sotto-sequenza dell'ultimo match, nonché un conteggio totale (`groupCount`) di tali sotto-sequenze.

Per comodità, vengono forniti anche metodi per restituire queste sottosequenze in forma di stringa (`group`)

≡ Esempi per la classe matcher

```
//Esempio N.1
Matcher matcher1 = pattern.matcher("dlkfIsASDaslsdSD"); //deve
corrispondere l'intera stringa
System.out.println(matcher1.matches());
Matcher matcher2 = pattern.matcher("dlkfIsASDaslsdSD 8798767"); //la
corrispondenza deve partire dall'inizio ma non è necessario che
corrisponda l'intera stringa
System.out.println(matcher2.lookingAt());
Matcher matcher3 = pattern.matcher("dlkfIsASDaslsdSD");
//cicla su tutti i matching
while (matcher3.find()) {
```

```

        System.out.println(matcher3.group() + ": " +
            matcher3.start() + "-" + matcher3.end());
    }
    //Esempio N.2
    //Una sequenza di numeri seguita da 1 o massimo 3 lettere min.
    String regexp = "([0-9]+)([a-z]{1,3})";
    Pattern pattern2 = Pattern.compile(regexp);
    String str = "9843989jf 39203920jie 32122i";
    Matcher matcher4 = pattern2.matcher(str);
    while (matcher4.find()) {
        //restituisce il numero di gruppi (individuati dalle parentesi
        //nella regexp)
        int gc = matcher4.groupCount();
        //il gruppo 0 corrisponde all'intero matching
        for (int i = 0; i <= gc; i++) {
            System.out.println(matcher4.group(i) + ": " +
                matcher4.start(i) + "-" + matcher4.end(i));
        }
        System.out.println();
    }
}

```

Collection

Una **collection** è un oggetto volto a racchiudere più oggetti al suo interno per poterli memorizzare, recuperare ed elaborare.

In parole povere, rappresenta un gruppo di cose che vanno tenute insieme, come:

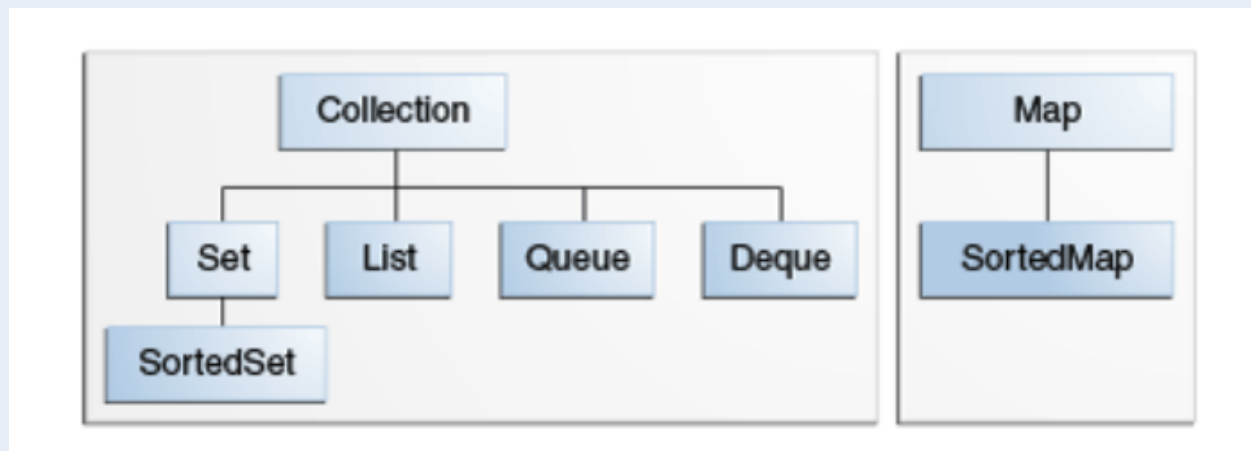
- Un mazzo di carte;
- L'elenco della rubrica telefonica;
- Insieme di email ricevute;
- etc...

Java mette a disposizione un **framework** (insieme di strumenti e librerie predefiniti che forniscono una struttura di base per lo sviluppo e la distribuzione di applicazioni in un particolare ambiente) per la gestione delle Collection, composto da:

- Interfacce
- Implementazioni
- Algoritmi (ricerca, ordinamento etc.) in grado di funzionare in modo polimorfo

Interfacce di collection

Gerarchia delle interfacce di Collection



Map è una collection particolare che non eredita da Collection

Ognuna di queste interfacce ha un diverso utilizzo:

- **Collection** è la radice della gerarchia e per questo la più generica, rappresenta semplicemente un contenitore di oggetti (elementi) senza alcun particolare vincolo
- **Set** rappresenta un contenitore di tipo insieme, non può contenere duplicati
 - **SortedSet** è un **Set** in cui gli elementi sono ordinati in ordine crescente
- **List** è una lista di elementi, ogni elemento avrà una posizione nella lista, ammette duplicati
- **Queue** è una coda in cui gli elementi hanno un preciso ordine di inserimento e recupero (si usa la tecnica FIFO, ma ci sono code particolari dette con priorità il cui ordine è dettato da una funzione di ordinamento)
- **Deque** è simile ad una coda ma permette l'accesso ad entrambe l'estremità della coda
- **Map** permette di collegare dei valori a delle chiavi (queste non possono essere duplicate all'interno della stessa **Map**)
 - **SortedMap** è una **Map** in cui le chiavi sono ordinate in ordine crescente

Metodi delle Collection generali

Tutte le interfacce che ereditano da Collection ereditano i suoi metodi primitivi per gestire un gruppo di oggetti:

- `int size()` restituisce il numero di oggetti
- `boolean isEmpty()` restituisce true se la Collection è vuota
- `boolean contains(Object element)` restituisce true se la collection contiene elementi
- `boolean add(E element)` aggiunge un elemento alla Collection
- `boolean remove(Object element)` rimuove un elemento alla Collection

- `Iterator iterator()` restituisce un oggetto `Iterator` che permette di iterare su tutti gli elementi della `Collection`

Alcuni metodi agiscono sull'intera `Collection`:

- `boolean containsAll(Collection c)` restituisce `true` se la `collection` contiene tutti gli elementi in `c`
- `boolean addAll(Collection c)` aggiunge tutti gli elementi in `c` alla `collection`
- `boolean removeAll(Collection c)` rimuove tutti gli elementi di `c` dalla `collection`
- `boolean retainAll(Collection c)` mantiene nella `collection` solo gli elementi presenti in `c`
- `void clear()` elimina tutti gli elementi dalla `collection`

L'interfaccia `Collection` ha due metodi `toArray` che permettono di fare da ponte tra le `collection` e gli array:

- `Object[] a = c.toArray()` trasforma la `collection c` in un array di oggetti, `a` avrà la stessa dimensione di `c`
- `String[] a = c.toArray(new String[0])`: se conosciamo il tipo di elementi in `c` possiamo creare un array che contiene gli stessi elementi di `c` e dello stesso tipo

Iterazione e interfaccia `Iterator`

Esistono due modi per iterare una `Collection`:

1. Il metodo `for-each`
2. Il metodo `iterator`, che utilizza l'interfaccia `Iterator` con i suoi diversi metodi tra cui
 - `hasNext()` restituisce `true` se ci sono altri elementi da visionare
 - `next()`: restituisce il prossimo elemento
 - `remove()`: rimuove l'elemento corrente

Classe `Iterator`

```
public interface Iterator {
    boolean hasNext();
    E next();
    void remove();
}
```

Filtrare elementi da una `collection`

```

Iterator it=collection.iterator()
while (it.hasNext()) {
    if (!cond(it.next())) it.remove(); //cond è un metodo fittizio
    che decide se un elemento rispetta o meno una particolare condizione
}

```

Nel caso quindi sia necessario rimuovere degli elementi è preferibile utilizzare un `Iterator` e non il metodo `for-each`

Set

Come detto precedentement, la sottoclasse `Set` rappresenta un contenitore di tipo insieme che non può contenere duplicati. Implementa tutti i metodi dell'interfaccia `Collection` e presenta tre implementazioni:

- **HashSet**: un set implementato da una tabella hash non mantiene l'ordine di inserimento degli elementi; è l'implementazione più efficiente
- **TreeSet**: un set implementato con una struttura ad albero che mantiene l'ordine di inserimento, è meno efficiente
- **LinkedHashSet**: un set implementato con una tabella hash e puntatore che mantiene l'ordine di inserimento degli elementi

L'uguaglianza degli oggetti in questa sottoclasse è definita dai metodi `equals` e `hashCode` (restituisce un intero) della classe `Object`

Siccome `Set` non può contenere duplicati, torna particolarmente utile nella creazione di una `Collection` senza duplicati partendo da una esistente

≡ Esempio di Collection senza duplicati

```
Set s=new LinkedHashSet(c);
```

s non conterrà duplicati

≡ Esempio di utilizzo di Set

```

public class Esempio1 {
    public static void main(String[] args) {

```

```
// Crea un nuovo Set (insieme) di stringhe utilizzando
LinkedHashSet.

Set<String> set = new LinkedHashSet<>();

// Aggiunge elementi al set.
set.add("a");
set.add("a"); // I duplicati vengono ignorati in un Set.
Questa operazione non ha effetto.
set.add("b");
set.add("c");

// Output: "3: [a, b, c]".
// La dimensione è 3 perché il secondo "a" non è stato
inserito.
System.out.println(set.size() + ": " + set);

// Rimuove l'elemento "a" dal set.
set.remove("a");

// Output: "2: [b, c]".
// Il set ora contiene solo 2 elementi.
System.out.println(set.size() + ": " + set);

// Crea un secondo Set chiamato set1 e ci aggiunge "b" e "c".
Set<String> set1 = new LinkedHashSet<>();
set1.add("b");
set1.add("c");

// removeAll elimina dal primo 'set' tutti gli elementi
presenti nel 'set1'.
// Dato che 'set' conteneva [b, c] e 'set1' contiene [b, c],
il primo set si svuota.
set.removeAll(set1);

// Output: "0: []".
// La dimensione è 0 e l'insieme è vuoto.
System.out.println(set.size() + ": " + set);
```

```
}
}
```

Quando si crea un Set, è possibile assegnare una variabile Set senza specificare quale si usi, per rendere il codice più generico possibile e non legarlo all'implementazione. Questo vale anche per tutte le implementazioni successive di `Collection`

Operazione sugli insiemi

Il set permette operazioni su insiemi, tra cui unione, intersezione e differenza

Unione

```
Set<Type> union = new HashSet<Type>(s1);
union.addAll(s2);
```

Il metodo `addAll(Collection c)` aggiunge tutti gli elementi della collezione specificata (in questo caso `s2`) all'insieme su cui viene chiamato, se non sono già presenti

Intersezione

```
Set<Type> intersection = new HashSet<Type>(s1);
intersection.retainAll(s2);
```

Il metodo `retainAll(Collection c)` mantiene nell'insieme corrente *solo* gli elementi che sono contenuti anche nella collezione specificata (`s2`). Rimuove tutto il resto.

Differenza

```
Set<Type> difference = new HashSet<Type>(s1);
difference.removeAll(s2);
```

Il metodo `removeAll(Collection c)` rimuove dall'insieme corrente tutti gli elementi che sono contenuti anche nella collezione specificata (`s2`).

Lista

Una lista è una sequenza di elementi ordinata, in questo caso sono ammessi duplicati (purchè si trovino in posizioni diverse) e sono presenti dei metodi oltre quelli previsti da Collection per sfruttare il suo ordinamento

Una lista non ha una dimensione predefinita, cresce dinamicamente

Implementazioni delle liste

Ci sono due implementazioni:

- `ArrayList` : lista basata sugli array, la più performante
- `LinkedList` : implementazioni con doppi puntatori

Metodi lista

Alcuni metodi sfruttano l'accesso alla posizione:

- `get(int i)` : restituisce l'oggetto alla posizione `i` (le posizioni partono da 0)
- `set(int i, E element)` : inserisce l'elemento alla posizione `i`
- `remove(int i)` : elimina l'oggetto in posizione `i`
- `add(int i, E element)` : aggiunge l'elemento in posizione `i`
- `addAll(int i, Collection c)` : aggiunge tutti gli elementi in `c` a partire dalla posizione `i`

Per ricercare degli elementi all'interno di una lista si usano questi metodi:

- `indexOf(Object o)` : restituisce la posizione in cui si trova l'oggetto `o`, altrimenti -1. Usa il metodo `equals` sotto.
- `lastIndexOf(Object o)` : restituisce l'ultima posizione in cui si trova l'oggetto `o`, altrimenti -1

`o` potrebbe trovarsi in più posizioni nella lista, se questo è duplicato viene restituita l'ultima posizione

Iterazione nella lista

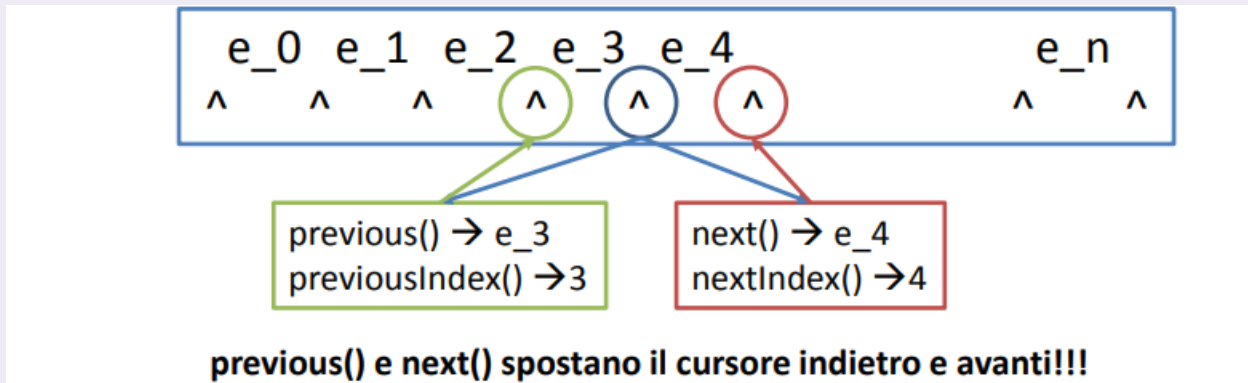
Per iterare gli elementi della lista esistono diversi metodi:

- `iterator()` : restituisce un iteratore su questa lista, gli elementi verranno elencati in sequenza
- Ci sono due metodi che restituiscono un `ListIterator` che permette di scorrere la lista sia in avanti che indietro:
 - `listIterator()` : restituisce un `ListIterator` che parte dall'inizio della lista
 - `listIterator(int i)` : restituisce un `ListIterator` che parte dalla posizione `i`

ListIterator

Il `ListIterator` è un iteratore che ammette un cursore tra un elemento e l'altro della lista (se non specificato si mette all'inizio ovviamente) e permette di andare all'elemento precedente `previous()` e all'elemento successivo `next()`

≡ ListIterator in forma grafica



`ListIterator` ha altri metodi implementati:

- `hasNext()` : booleano che controlla se è presente un elemento avanti
- `hasPrevious()` : booleano che controlla se è presente un elemento indietro
- `remove()` : rimuove l'elemento ottenuto dall'ultima chiamata di `next()` o `previous()`
- `add(E e)` : aggiunge un elemento tra `previous()` e `next()`
- `set(E e)` : sostituisce con `e` l'elemento ottenuto dall'ultima chiamata di `next()` o `previous()`

≡ Esempio di operazioni base su una lista

```
public class List1 {
    public static void main(String[] args) {
        // Inizializza una nuova ArrayList di tipo String
        List<String> list = new ArrayList<>();

        // Aggiunge elementi in coda alla lista
        list.add("a");
        list.add("b");
        list.add("c");
        list.add("a");
        System.out.println(list); // Stampa: [a, b, c, a]

        // Sostituisce l'elemento all'indice 0 ("a") con "z"
        list.set(0, "z");
        System.out.println(list); // Stampa: [z, b, c, a]
    }
}
```

```

        // Inserisce "d" esattamente all'indice 3, spostando in avanti
        gli elementi successivi
        list.add(3, "d");
        System.out.println(list); // Stampa: [z, b, c, d, a]

        // Recupera e stampa l'elemento all'indice 1 ("b")
        System.out.println(list.get(1));

        // Rimuove l'elemento all'indice 2 (che attualmente è "c")
        list.remove(2);
        System.out.println(list); // Stampa: [z, b, d, a]

        // Cerca e stampa l'indice della prima occorrenza di "a"
        System.out.println(list.indexOf("a")); // Stampa: 3

        // Aggiunge un'altra "a" in fondo e cerca l'indice della sua
        ultima occorrenza
        list.add("a");
        System.out.println(list.lastIndexOf("a")); // Stampa: 4
    }
}

```

☰ Esempio di ListIterator

```

public class List2 {
    public static void main(String[] args) {
        // Crea e popola la lista
        List<String> list = new ArrayList<>();
        list.add("a");
        list.add("b");
        list.add("c");
        list.add("a");
        list.add("z");
        list.add("h");

        // Crea un ListIterator associato alla lista
        ListIterator<String> lit = list.listIterator();
    }
}

```



```

        // hasNext() verifica se c'è un elemento successivo nella
        lista
        while (lit.hasNext()) {
            // Stampa l'indice precedente (-1 alla prima iterazione) e
            l'indice successivo
            System.out.print(lit.previousIndex() + "\t" +
            lit.nextIndex() + " -> ");

            // next() recupera l'elemento corrente e sposta il cursore
            in avanti
            System.out.println(lit.next());
        }
    }
}

```

☰ Esempio di scorrimento all'indietro

```

public class List3 {
    public static void main(String[] args) {
        // Crea e popola la lista
        List<String> list = new ArrayList<>();
        list.add("a");
        list.add("b");
        list.add("c");
        list.add("a");
        list.add("z");
        list.add("h");

        // Inizializza l'iteratore partendo dal PENULTIMO elemento
        (size - 1)
        ListIterator<String> lit = list.listIterator(list.size() - 1);

        // hasPrevious() verifica se ci sono elementi precedenti
        rispetto al cursore
        while (lit.hasPrevious()) {
            // previous() recupera l'elemento e sposta il cursore
            all'indietro

```

```

        System.out.println(list.previous());
    }
}

```

Sotto lista

Il metodo `subList(int start, int end)` permette di ottenere una sottolista a partire dalla posizione start fino alla posizione end (non inclusa).

`subList()` restituisce una nuova lista con gli elementi richiesti, non viene fatta nessuna modifica sulla lista di partenza

Algoritmi sulle liste

La classe `Collections` (da non confondere con quella classica `Collection`) mette a disposizione dei metodi statici per effettuare degli algoritmi sulle classi che implementano `Collection`:

- sort: ordina gli elementi nella lista
- shuffle: permuta in modo casuale gli elementi nella lista
- reverse: inverte l'ordine degli elementi
- rotate: ruota gli elementi di una lista
- swap: scambia due elementi nella lista
- replaceAll: sostituisce un elemento con un altro valore
- fill: sostituisce tutti gli elementi con un altro valore
- copy: copia tra due liste
- binarySearch: ricerca un elemento nella lista utilizzando la ricerca binaria
- indexOfSubList, lastIndexOfSubList: cercano una sottolista all'interno di una lista

☰ Esempio di shuffling

```

public class List4 {
    public static void main(String[] args) {
        // Inizializzazione della lista con un nome più chiaro
        List<String> list = new ArrayList<>();

        // Aggiunta degli elementi
        list.add("pippo");
        list.add("topolino");
        list.add("paperino");
    }
}

```

```

// Stampa iniziale: [pippo, topolino, paperino]
System.out.println(list);

// Ordina la lista in ordine alfabetico
Collections.sort(list);
System.out.println(list); // [paperino, pippo, topolino]

// Mescola gli elementi in modo casuale
Collections.shuffle(list);
System.out.println(list);

// Ruota gli elementi. Con -1, il primo elemento diventa
l'ultimo
Collections.rotate(list, -1);
System.out.println(list);
    }
}

```

SortedSet

Visione veloce di SortedSet

```

public interface SortedSet<E> extends Set<E>
{
    // Range-view
    SortedSet<E> subSet(E fromElement, E
toElement);
    SortedSet<E> headSet(E toElement);
    SortedSet<E> tailSet(E fromElement);

    // Endpoints
    E first();
    E last();

    // Comparator access
    Comparator<? super E> comparator();
}

```

Gli elementi sono ordinati in modo decrescente in base al loro ordinamento naturale o può essere passato un Comparator nel costruttore

Le range-view permettono viste sugli oggetti sfruttando il loro ordinamento

Il primo e l'ultimo elemento del set

Permette di accedere al Comparator di questo set

☰ Ricerca frequenza parole con SortedSet

```
import java.util.HashMap;
import java.util.Map;

public class Freq {
    public static void main(String[] args) {
        // Inizializza una nuova HashMap.
        // La Chiave (String) sarà la parola, il Valore (Integer) sarà
        // il suo conteggio.
        Map<String, Integer> m = new HashMap<>();

        // Inizializza la tabella delle frequenze leggendo dalla riga
        // di comando (args)
        for (String a : args) {
            // Cerca la parola 'a' nella mappa.
            // Se la parola non è ancora stata inserita, m.get()
            // restituirà null.
            Integer freq = m.get(a);

            // L'operatore ternario (condizione ? caso_vero :
            // caso_falso) decide cosa inserire:
            // - Se freq è null (parola nuova), inserisce la parola
            // con valore 1.
            // - Se freq NON è null (parola già vista), reinserisce la
            // parola
            // sovrascrivendo il vecchio valore con freq + 1.
            m.put(a, (freq == null) ? 1 : freq + 1);
        }

        // m.size() restituisce il numero di chiavi (ovvero le parole
        // uniche)
        System.out.println(m.size() + " distinct words:");

        // Stampa il contenuto dell'intera mappa nel formato
        // {chiave1=valore1, chiave2=valore2...}
        System.out.println(m);
    }
}
```

```
}
}
```

Queue

L'interfaccia `Queue`, come già detto precedentemente, crea una coda per poter mantenere degli elementi e processarli in un preciso ordine (di solito FIFO).

Essendo un'interfaccia, non è possibile istanziare un oggetto di tipo `Queue`, ma estende una `Collection` già presente.

Generalmente questa interfaccia implementa dei metodi che creano delle eccezioni o fanno il return di un valore specifico (null o false)

Metodi in Queue

- `add(E e)` : aggiunge un elemento alla coda, restituisce true se l'elemento è inserito altrimenti genera un'eccezione
- `element()` : restituisce l'elemento in testa alla coda senza rimuoverlo, se la coda è vuota genera un'eccezione
- `offer(E e)` : aggiunge un elemento alla coda, in caso contrario offre valore null
- `peek()` restituisce l'elemento in testa alla coda senza rimuoverlo, oppure null se la coda è vuota
- `poll()` : restituisce e rimuove l'elemento in testa alla coda, null se la coda è vuota
- `remove()` : restituisce e rimuove l'elemento in testa alla coda, se la coda è vuota genera un'eccezione

I metodi di inserimento, cancellazione e recupero sono duplicati e si differenziano sul loro comportamento (eccezione/restituzione valore) nel caso in cui la coda è vuota o ha raggiunto la capacità massima

☰ Esempio operazioni base

```
public class Queue1 {
    public static void main(String[] args) {
        // Crea una nuova coda FIFO (First-In-First-Out) basata su
        LinkedList
        Queue<String> q = new LinkedList<>();

        // offer() aggiunge un elemento in fondo alla coda.
        q.offer("g");
        q.offer("h");
        q.offer("a");
    }
}
```

```

        // peek() recupera il primo elemento della coda.
        // Poiché "g" è stato inserito per primo, sarà il primo della
        fila.
        System.out.println(q.peek()); // Stampa: g
    }
}

```

Implementazioni di Queue

L'implementazioni di default di una coda è quella di `LinkedList`, che implementa sempre la coda FIFO. Nel caso ci sia bisogno di una lista con priorità, si usa `PriorityQueue`, l'elemento sempre in cima è quello più piccolo.

☰ Esempio coda con priorità

```

public class Queue2 {
    public static void main(String[] args) {
        // Inizializza una PriorityQueue. Gli elementi verranno
        ordinati
        // automaticamente in base alla loro "priorità" (ordine
        alfabetico).
        Queue<String> q = new PriorityQueue<>();

        // Aggiunge elementi alla coda
        q.offer("g");
        q.offer("h");
        q.offer("a");

        // peek() guarda il primo elemento in coda, .
        // Poiché "a" viene prima di "g" e "h" nell'alfabeto, è lui
        // l'elemento con la priorità più alta.
        System.out.println(q.peek()); // Stampa: a
    }
}

```

Deque (Deck)

Deque è una Collection che è simile alla coda, ma con l'eccezione di poter inserire elementi all'inizio e alla fine di essa.

Metodi di Deque

Rispetto alla coda, Deque implementa le sue controparti:

- `addFirst(E e)`, `addLast(E e)` : funziona come la controparte di Queue, una aggiunge in cima alla coda e il secondo alla fine delle coda (fa risultare un eccezione nel caso non sia possibile)
- `offerFirst(E e)`, `offerLast(E e)` funziona come la controparte di Queue, una aggiunge in cima alla coda e il secondo alla fine delle coda (valore null nel caso non sia possibile)
- `removeFirst()`, `removeLast()` funziona come la controparte di Queue, una rimuove in cima alla coda e il secondo alla fine delle coda (fa risultare un eccezione nel caso non sia possibile)
- `pollFirst()`, `pollLast()` funziona come la controparte di Queue, una rimuove in cima alla coda e il secondo alla fine delle coda (valore null nel caso non sia possibile)
- `getFirst()`, `getLast()` funziona come la controparte di Queue, una prende il valore in cima alla coda e il secondo alla fine della coda (fa risultare un eccezione nel caso non sia possibile)
- `peekFirst()`, `peekLast()` funziona come la controparte di Queue, una prende il valore in cima alla coda e il secondo alla fine della coda (valore null nel caso non sia possibile)

≡ Esempio di Deque

```
public class Deque1 {
    public static void main(String[] args) {
        // Inizializza una Deque utilizzando l'implementazione
        LinkedList.
        Deque<String> q = new LinkedList<>();

        // offerLast() aggiunge elementi in coda (alla fine della
        lista)
        q.offerLast("g"); // Stato attuale: [g]
        q.offerLast("h"); // Stato attuale: [g, h]

        // offerFirst() aggiunge un elemento in testa (all'inizio
        della lista),
        // spingendo indietro gli elementi già presenti.
        q.offerFirst("z"); // Stato attuale: [z, g, h]
```

```

// Aggiunge un altro elemento in coda
q.offerLast("a"); // Stato attuale: [z, g, h, a]

// peekLast() legge l'ultimo elemento (senza rimuoverlo)
System.out.println("Coda: " + q.peekLast()); // Stampa:
a

// peekFirst() legge il primo elemento (senza rimuoverlo)
System.out.println("Testa: " + q.peekFirst()); // Stampa:
z
    }
}

```

Map

L'interfaccia MAP permette di creare delle associazioni chiave-valore. Ogni chiave è univoca e ad ognuna di essa è associato un solo valore.

Esistono tre implementazioni dell'interfaccia Map (simili a Set):

- HashMap
- TreeMap
- LinkedHashMap

Metodi di Map

L'interfaccia Map implementa alcuni metodi utili

- `get(Object key)` : restituisce l'oggetto associato alla chiave `key`
- `put(K key, V value)` : inserisce una coppia chiave-valore nella `Map`
- `remove(Object key)` : rimuove la coppia chiave-valore associata a `key`
- `containsKey(Object key)` , : booleano che restituisce true se la Map contiene la chiave `key`
- `containsValue(Object value)` : booleano che restituisce true se la Map contiene il valore `value`
- `putAll(Map<K, V> map)` : inserisce tutti gli elementi di `map`

☰ **Calcolare le frequenze con una HashMap**


```

public class Freq {
    public static void main(String[] args) {
        // Inizializza una nuova HashMap.
        // La Chiave (String) sarà la parola, il Valore (Integer)
        // sarà il suo conteggio.
        Map<String, Integer> m = new HashMap<>();

        // Inizializza la tabella delle frequenze leggendo dalla
        // riga di comando (args)
        for (String a : args) {
            // Cerca la parola 'a' nella mappa.
            // Se la parola non è ancora stata inserita, m.get()
            // restituirà null.
            Integer freq = m.get(a);

            // L'operatore ternario (condizione ? caso_vero :
            // caso_falso) decide cosa inserire:
            // - Se freq è null (parola nuova), inserisce la
            // parola con valore 1.
            // - Se freq NON è null (parola già vista),
            // reinserisce la parola
            // sovrascrivendo il vecchio valore con freq + 1.
            m.put(a, (freq == null) ? 1 : freq + 1);
        }

        // m.size() restituisce il numero di chiavi (ovvero le
        // parole uniche)
        System.out.println(m.size() + " distinct words:");

        // Stampa il contenuto dell'intera mappa nel formato
        // {chiave1=valore1, chiave2=valore2...}
        System.out.println(m);
    }
}

```

SortedMap

📄 Visione veloce di SortedMap

```
public interface SortedMap<K, V>
    extends Map<K, V>{
    Comparator<? super K>
    comparator();
```

Gli elementi sono ordinati in modo decrescente in base all'ordinamento naturale delle chiavi o può essere passato un Comparator nel costruttore

```
    SortedMap<K, V> subMap(K
    fromKey, K toKey);
    SortedMap<K, V> headMap(K
    toKey);
    SortedMap<K, V> tailMap(K
    fromKey);
```

Le range-view permettono viste sugli oggetti sfruttando l'ordinamento sulle chiavi

La prima e l'ultima chiave

```
    K firstKey();
    K lastKey();
}
```

Equals e hashCode

Ogni classe eredita da Object due importanti metodi:

- `equals()`, che ricordiamo essere il metodo per confrontare due operatori, restituisce un valore booleano `true` quando due oggetti sono uguali
Il metodo `equals` è utilizzato da Set per evitare duplicati, ma anche da tutti i metodi di Collection che cercano un oggetto
- `hashCode()`, che restituisce il codice intero hash per l'oggetto. È utilizzato dalle tabelle hash come quelle fornite da HashMap

Se due oggetti sono uguali in base al metodo `equals (Object)`, la chiamata del metodo `hashCode` su ciascuno dei due oggetti deve produrre gli stessi risultati interi, poichè Object genera l'hash code utilizzando l'indirizzo di memoria dell'oggetto

Generics

Le Collection sono originariamente strutture generiche nate per memorizzare oggetti di tipo generico `Object`, tuttavia è possibile tipizzare queste collezioni utilizzando i generics attraverso la sintassi `<T>`, la quale permette a tutti i metodi della struttura di lavorare specificamente su oggetti di tipo `T` anziché sul tipo generico `Object`

Per comprendere l'utilità dei generics, è fondamentale analizzare come cambia la gestione dei tipi di dato nel codice:

☰ Senza vs Con Generics

Senza generics

```
List list = new ArrayList();
list.add("hello");
String s = (String) list.get(0);
```

E' necessaria l'operazione di cast (cambio del tipo di oggetto)

Con generics

```
List<String> list = new ArrayList<String>();
list.add("hello");
String s = list.get(0); // no cast
```

Il metodo get è stato tipizzato in base al generics <String>

Eccezioni

Le eccezioni sono eventi che si verificano durante l'esecuzione del programma e identifica un'anomalia che impedisce il normale flusso del programma.

In Java esiste un meccanismo (gestore delle eccezioni) che cattura e gestisce le eccezioni, che di solito possono essere generate durante l'esecuzione di un metodo. Il gestore delle eccezioni gestisce soltanto un determinato tipo di eccezione.

Ogni eccezione ha un oggetto con tutte le informazioni su cosa è accaduto.

Tipologie di eccezioni

Esistono 3 grandi tipologie di eccezioni:

- **Exception (con sottoclassi)**: sono eccezioni che possono essere gestite e catturate (anche se il programmatore dovrebbe anticipare e gestire questo tipo di eccezioni)
- **Error (e sottoclassi)**: sono eccezioni che non possono essere gestite e dipendono da qualcosa esterno al programma
- **RuntimeException (e sottoclassi)**: eccezioni interne al programma che non possono essere anticipate o catturate (essenzialmente sono dei bug)

Tutte le eccezioni sono sottoclassi di Exception, ed è possibile creare una propria classe exception, estendendo quella originale o una delle sue sottoclassi

Try-catch and finally

Per catturare un'eccezione bisogna utilizzare il blocco try-catch

Blocco try-catch

```
try {
    //istruzioni
} catch (Exception ex) {
    //gestione dell'eccezione
} finally {
    //istruzioni che devono essere indipendentemente eseguite dal
    //successo o non successo della try-catch
}
```

Il blocco `finally` è opzionale, può essere utilizzato per effettuare delle operazioni di "pulizia"

Per gestire più eccezioni nella stessa linea di codice si usa la seguente sintassi:

Catch di più eccezioni

```
try {
    //Istruzioni
} catch (IOException | SQLException ex) {
    System.err.println("Caught Exception: " + ex.getMessage());
}
```

`System.err.println()` stampa l'errore ma non ferma l'esecuzione del programma.

Lancio di un'eccezione

I metodi possono dichiarare il tipo di eccezione che si potrebbe verificare al suo interno. Questo in sostanza obbliga tutti i metodi chiamanti a gestire l'eccezione nel blocco try-catch

Esempio di lancio di un'eccezione

```
public File openFile(String filename) throws IOException{
    try {
        //istruzioni
    } catch (IOException ex) {
```

```

        throw ex;
    }
}

```

Il `throw` generalmente si usa per imporre regole rigide (come dei parametri importanti non validi, file non trovati, connessioni perse) in modo da interrompere il flusso di esecuzione. Una guida interessante con esempi ancora più chiari la trovate [qui](#)

I/O Stream

Un flusso I/O rappresenta una sorgente di input o una destinazione di output e sono di vario tipo come
file su disco, array di memoria...

Gli **stream** supportano molti tipi diversi di dati, dai byte semplici agli oggetti. Alcuni flussi si occupano di **trasmettere dati**, mentre altri li **manipolano**. Indipendentemente da come funzionano, tutti i flussi presentano **una sequenza di dati**.

Byte stream

I byte stream sono utilizzati per leggere e scrivere byte (8 bit) su un dispositivo di I/O. Ereditano dalle classi `InputStream` e `OutputStream`.

Ci sono diverse classi che ereditano direttamente dai byte stream per scrivere e leggere sui file:

- `FileInputStream`
- `FileOutputStream`

≡ Esempio di scrittura e output di un file

Ad esempio:

```

public class CopyBytes{
    public static void main(String[] args) throws IOException{
        FileInputStream in = null;
        FileOutputStream out = null;
        try{
            in = new FileInputStream("sorgente.txt");
            out = new FileOutputStream("destinazione.txt");
            int c;
            while((c = in.read())!= -1)
                out.write(c);
        }
    }
}

```

```

    }
    finally{
        if(in != null)
            in.close();
        if(out != null)
            out.close();
    }
}
}

```

Si **chiudono sempre** gli stream. È di vitale importanza eseguire questo, poiché possiamo de-allocare risorse e evitiamo di perdere le informazioni scritte sullo stream.

Lo stream sui byte andrebbe evitato poiché **non è una operazione adeguata al compito**, essa infatti è un'operazione di basso livello ed esistono degli stream idonei per scrivere e leggere caratteri per caratteri.

Character stream

Il character stream si usa in caso dovessimo eseguire operazioni IO di caratteri. Questo tipo di flusso gestisce automaticamente la codifica corretta per i caratteri, seguendo lo stile UTF o ISO.

Le classi per questo tipo di stream sono:

- `FileReader`
- `FileWriter`

Esempio di character stream

Ad esempio:

```

FileReader inputStream = null;
FileWriter outputStream = null;

try{
    inputStream = new FileReader("sorgente.txt");
    outputStream = new FileWriter("destinazione.txt");

    int c;
    while ((c = inputStream.read()) != -1){

```

```

        outputStream.write(c);
    }
} finally{
    if(inputStream != null){
        inputStream.close();
    }
    if(outputStream != null){
        outputStream.close();
    }
}
}

```

Line-oriented

Per facilitare il lavoro in un file pieno di caratteri (e generalmente i file presentano più testi che caratteri), si può raggruppare per testi.

Per poterlo implementare in questa maniera dobbiamo accedere a sequenze di caratteri, ovvero **prelevare le stringhe**.

Le classi I/O che permettono il prelievo di una stringa e l'inserimento di una stringa su un file sono:

- BufferedReader
- PrintWriter

≡ Esempio di stream line oriented

```

BufferedReader inputStream = null;
PrintWriter outputStream = null;

try{
    inputStream = new BufferedReader(new
    FileReader("sorgente.txt"));
    outputStream = new PrintWriter(new
    FileWriter("destinazione.txt"));

    String l;
    while ((l = inputStream.readLine())!=null){
        outputStream.println(l);
    }
} finally{

```

```

        if(inputStream != null){
            inputStream.close();
        }
        if(outputStream != null){
            outputStream.close();
        }
    }
}

```

Buffered I/O

Il Buffered I/O è il più famoso ed utilizzato, specialmente per lo sviluppo in Java. E' molto conveniente poiché le operazioni vengono eseguite **direttamente sul dispositivo di I/O**; questo è possibile poiché le classi di Java (generalmente) sono **unbuffered**.

Buffered e Unbuffered

- **Unbuffered Stream:** Ogni singola richiesta di lettura o scrittura viene gestita e inviata **direttamente e immediatamente al dispositivo di I/O** (come il disco rigido)
- **Buffered Stream:** Utilizza un'area di memoria temporanea e velocissima nella RAM (chiamata buffer)

Il confronto tra questi due metodi serve a dimostrare il principio dell'**ottimizzazione delle prestazioni**, generalmente il collo di bottiglia è la scrittura su supporto fisico o le velocità di trasmissioni di rete

Le classi buffered utilizzano un buffer predisposto all'I/O che velocizza le operazioni di lettura e scrittura.

Le funzioni che seguono questa classe sono:

- **BufferedInputStream** e **BufferedOutputStream**: creano le versioni buffered di un byte stream
- **BufferedReader** e **BufferedWriter**: creano le versioni buffered di un character stream

A livello di codice si creano nel seguente modo:

Esempio di creazione BufferedReader e BufferedWriter


```
inputStream = new BufferedReader(new FileReader("pippo.txt"));
outputStream = new BufferedWriter(new FileWriter("pluto.txt"));
```

La classe File

La classe `File` può rappresentare due concetti diversi sotto lo stesso nome:

- **nome** di un particolare file
- **nome** di una directory

Nell'ultimo caso possiamo conoscere gli insieme dei file che lo compongono tramite il metodo `list()`, che restituisce un array di stringe con gli elementi di tale insieme.

È possibile anche selezionare solo un tipo di oggetti nella cartella ricorrendo ad un **filtro**, detto **directory filter**.

L'interfaccia `FilenameFilter` è la seguente:

```
public interface FilenameFilter{ boolean accept (File dir, String name);}
```

Le classi che implementano questa funzione devono fornire sia un metodo `accept()` obbligatoriamente sia un metodo `list()` della classe madre `File`, così da eseguire una **call back** per determinare quali nomi di file devono essere inclusi nella lista.

Metodo accept()

Gli argomenti del metodo `accept()` sono due:

- Un oggetto **File** che rappresenta la directory in cui si trova il file
- Un oggetto **String** che rappresenta il nome del file

Esiste un'altra interfaccia simile chiamata `FileFilter` in cui il metodo `accept` prende in input direttamente l'oggetto `File`.

La classe `File` è usata anche per:

- creare nuove cartelle o percorsi tramite `\mkdir()` o `\mkdirs()`
- accedere alle caratteristiche dei file
- eliminare un file
- verificare l'oggetto `File`

I/O con l'utente

Ogni output visto fino a questo momento non è formattato adeguatamente. Per adeguarci a questa necessità abbiamo bisogno di poter formattare l'output in modo da renderlo

comprensibile, così da poter ottenere le singole informazioni necessarie dall'input di un utente

☰ Esempio di input formattato tramite Scanner

```
Scanner s = null;
double sum = 0;
try{
    s = new Scanner(new BufferedReader(new FileReader("input.txt")));
    while (s.hasNext()){
        if( s.hasNextDouble()){ //ricerchiamo il dato semplice
            Double
                sum+=s.nextDouble();
        } else s.next();
    }
} finally{
    s.close();
}
System.out.println(sum);
```

La classe `Scanner` permette quindi di processare l'input suddividendolo i token e traducendolo rispetto ad un tipo predefinito. I token vengono suddivisi utilizzando i white space.

`new FileReader("input.txt")` prende i caratteri direttamente dal file sul disco, `new BufferedReader(...)` li accumula in un buffer situato in RAM.

I/O da linea di comando

I/O da linea di comando

La classe `System` mette a disposizione tre stream collegati al terminale:

- `System.in`: `InputStream` che legge l'input
- `System.out`: `PrintStream` che stampa l'output
- `System.err`: `PrintStream` che stampa messaggi di errore

☰ Esempio di I/O con l'utente da linea di comando

```
public static void main(String args[]){
    Scanner scanner=new Scanner(new InputStreamReader(System.in));
```

```

String s= "";
while (scanner.hasNext()){
    s= scanner.next();
    if(!s.equalsIgnoreCase("exit")){
        System.out.println("Hai scritto: "+s);
    } else{
        System.out.println("Goodbye!");
        break;
    }
}
scanner.close();
}

```

Data Streams

I flussi di dati supportano l'I/O binario dei valori di dati primitivi, tra cui le stringhe pure.

I flussi di dati implementano l'interfaccia **DataInput** o **DataOutput**, tra cui le più utilizzate di queste due interfacce sono proprio **DataInputStream** e **DataOutputStream**.

L'esempio **DataStreams** mostra i flussi di dati scrivendo un **set di record di dati** e quindi rilegendoli. Ogni record è costituito da tre valori relativi ad un articolo:

- Prezzo (double),
- quantità (int),
- descrizione (String).

≡ Esempio di Datastream

```

static final String dataFile="invoicedata";

static final double[] prices={19.99,9.99,15.99,4.99};
static final int[] units={12,8,13,29,50};
static final String[] desc={
    "Java T-shirt",
    "Java Mug",
    "Duke Juggling Dolls",
    "Java Pin",
};

```

`DataStream` rileva una condizione di fine file catturando una **`EOFException`**, anziché testare un valore restituito non valido, come i modelli visti precedentemente. Generalmente si usa **`IOException (end of stream)`**.

Ogni scrittura in `DataStream` corrisponde esattamente alla lettura corrispondente passo passo. Il programmatore deve assicurarsi che i tipi di output ed input siano abbinati correttamente. I dati in input sono binari, senza nulla che indichi il tipo dei singoli valori o da dove inizia il flusso.

Serializzazione degli oggetti

Se si volesse salvare dei dati su un file che non siano di tipo primitivo, come gli oggetti, `DataStream` da solo non basta. Dovremmo memorizzare tutte le parti di un oggetto in maniera separata, con una rappresentazione ben precisa così da ricostruire l'informazione correttamente al momento del bisogno. Questo processo prende il nome di **persistenza**.

Definizione di persistenza

La **persistenza** di un oggetto è la capacità dell'oggetto di vivere separatamente dal programma che lo ha generato

Java contiene un meccanismo interno per creare oggetti persistenti, detto **serializzazione**. La serializzazione trasforma in una sequenza di byte il concetto che vogliamo rappresentare, dopo di che questa rappresentazione di byte può essere ricostruita nel suo aspetto originale. Dopo che l'oggetto viene serializzato può essere salvato su un file o inviato ad un altro PC.

La sua realizzazione prevede l'uso di un'interfaccia con due classi:

- `ObjectOutputStream`, il quale contiene il metodo `writeObject` che serve per serializzare un oggetto.
- `ObjectInputStream`, il quale contiene il metodo `readObject` che serve per deserializzare un oggetto.

Ogni oggetto che si vuole serializzare deve implementare l'interfaccia `Serializable`, la quale non contiene metodi e serve soltanto al compilatore per comprendere che un oggetto di quella determinata classe può essere serializzato.

`ObjectInputStream` e `ObjectOutputStream` sono **stream di manipolazione** e devono essere utilizzati congiuntamente a un `OutputStream` e un `InputStream`.

Esempio di serializzazione

```
FileOutputStream outFile = new FileOutputStream("info.dat");
ObjectOutputStream outStream = new ObjectOutputStream (outFile);
```

```
outStream.writeObject(myCar); // dove myCar è un oggetto di una classe
Car definita dal programmatore
```

Per poter leggere l'oggetto serializzato e ricaricarlo in memoria centrale si procederà come segue:

```
FileInputStream inFile = new FileInputStream("info.dat");
ObjectInputStream inStream = new ObjectInputStream(inFile);
Car myCar = (Car) inStream.readObject();
```

La serializzazione di un oggetto si occupa di serializzare tutti gli eventuali riferimenti ad esso collegati. Se la classe Car contenesse dei riferimenti (variabili di classe o di istanza) a oggetti di classe Engine, questa verrebbe serializzata automaticamente e diverrebbe parte della serializzazione di Car. La classe Engine dovrà, pertanto, implementare anch'essa l'interfaccia serializable al suo interno.

⚡ Errore comune

Gli attributi di classe, cioè definiti come static, **non vengono serializzati**. Per poterli salvare occorre provvedere in modo personalizzato.

≡ Esempio di serializzazione personalizzata

```
class Nave implements Serializable {

    private static int nroNavi = 1;
    private int nroNave;
    private String nomeNave;

    // Costruttore: assegna il numero progressivo e il nome alla nave
    Nave(String nomeNave) {
        nroNave = nroNavi++; // Assegna il numero corrente e incrementa
        // il contatore
        this.nomeNave = nomeNave; // Assegna il nome passato come
        // parametro
    }

    // Restituisce una rappresentazione testuale della nave
    public String toString() {
```

```

        return nomeNave + ": " + nroNave;
    }

    // Metodo per SALVARE (serializzare) l'oggetto su file binario
    "info.dat"

    public void salva() throws FileNotFoundException, IOException {
        ObjectOutputStream out = new ObjectOutputStream(new
        FileOutputStream("info.dat"));
        out.writeObject(this); // Serializza l'oggetto Nave corrente
        out.writeObject(nroNavi);
        out.close();
    }

    // Metodo statico per CARICARE (deserializzare) un oggetto Nave
    dal file "info.dat"

    public static Nave carica() throws FileNotFoundException,
    IOException {
        ObjectInputStream in = new ObjectInputStream(new
        FileInputStream("info.dat"));
        Nave n = (Nave) in.readObject(); // Legge e ricostruisce
        l'oggetto Nave
        Nave.nroNavi = in.readObject();
        in.close();
        return n; // Restituisce la nave deserializzata
    }
}

```

Molte classi della **libreria standard Java** implementano l'interfaccia **Serializable** in modo da essere serializzate quando necessario, come ad esempio la classe String.

Transient

A volte, quando si serializza un oggetto, si può desiderare di **escludere delle informazioni**, come una password.

Questo accade quando le informazioni vengono trasmesse via rete, poiché il pericolo è che, pur dichiarandola con visibilità privata, la password possa essere letta e usata da soggetti non autorizzati quando viene serializzata.

Per questo tipo di richiesta e problematica ci viene in aiuto la parola chiave **transient** associato al nome di una variabile, esso indica al compilatore di nascondere quella variabile

così da non rappresentarla come parte dello stream di byte della versione serializzata dell'oggetto.

≡ Esempio di variabile transient

```
private transient String password;
private String CF;
```

In questo caso, durante la serializzazione dell'oggetto che include queste due variabili, la variabile CF, anche se privata, sarà serializzata, mentre la password sarà ignorata nella rappresentazione grazie alla keyword transient.

Generics

Le Generics in Java vengono introdotte per poter superare il limite imposto dalle interfacce nella generalizzazione (ossia il **polimorfismo per inclusione**)

≡ Limitazione delle interfacce in Java

È possibile implementare un metodo m che prende in input un oggetto della classe base A come argomento. Tale metodo potrà essere utilizzato su tutte le sottoclassi della classe A (ammesso che A non sia dichiarata `final`).

La limitazione della applicabilità del metodo m ad un albero di ereditarietà con radice A può essere superato con l'utilizzo di un'interfaccia, ma alcune volte anche questo diventa un limite, dovendo ricorrere all'astrazione generica

Le Generics quindi permettono un astrazione maggiore rispetto alle interfacce, creando **classi contenitori** in grado di contenere oggetti omogenei di qualsiasi classe

≡ Esempio di Generics (W3Schools)

```
class Box<T> {
    T value; // T is a placeholder for any data type

    void set(T value) {
        this.value = value;
    }

    T get() {
        return value;
    }
}
```

```

}

public class Main {
    public static void main(String[] args) {
        // Create a Box to hold a String
        Box<String> stringBox = new Box<>();
        stringBox.set("Hello");
        System.out.println("Value: " + stringBox.get());

        // Create a Box to hold an Integer
        Box<Integer> intBox = new Box<>();
        intBox.set(50);
        System.out.println("Value: " + intBox.get());
    }
}

```

Uno degli scopi primari delle Java Generics risiede nella capacità di fornire un meccanismo di casting automatico e sicuro al momento dell'istanziamento di un tipo parametrizzato, delegando al compilatore il compito di verificare la consistenza dei tipi rigorosamente a tempo di compilazione (compile-time)

Tuple di Generics

In alcuni casi è possibile che sia necessario definire una funzione che restituisca non un singolo valore ma una coppia di valori o una tripla, allora si può definire con le Generics delle tuple

☰ Esempio di tuple in Generics

```

public class TwoTuple<A,B> {
    public final A first;
    public final B second;
    public TwoTuple(A a, B b) { first = a; second = b; }
    public String toString() {
        return "(" + first + ", " + second + ")";
    }
}

```



```
TwoTuple tt = new TwoTuple("hi", 47);  
System.out.println(tt.toString());
```

Containers per Generics

Le Generics sono basilari per l'implementazione di tipi astratti di dati classici, come lo **stack**. È possibile strutturare uno stack dinamico senza vincolarlo al tipo di dato memorizzato, impiegando, ad esempio, una classe interna privata (inner class) destinata alla rappresentazione del singolo nodo della struttura

☰ Esempio di containers

```

public class LinkedStack<T> {

    private class Node<U> {
        U item;
        Node<U> next;
        Node() { }
        Node(U item, Node<U> next) {
            this.item = item;
            this.next = next;
        }
        boolean end() {
            return item == null && next == null; }
    }

    private Node<T> top = new Node<T>();

    public void push(T item) {
        top = new Node<T>(item, top);
    }

    public T pop() {
        T result = top.item;
        if(!top.end())
            top = top.next;
        return result;
    }
}

```



Node<U> è una classe interna a LinkedStack<T>. Questo significa che può essere utilizzata all'interno di LinkedStack<T>. Potrebbe anche essere visibile all'esterno, ma in questo caso è definita privata, quindi non è visibile all'esterno.

La classe Node<U> è di servizio alla definizione di LinkedStack<T>, per questa ragione non è resa accessibile se non a quest'ultima.

In generale non è possibile creare un oggetto di una classe interna a meno che non si disponga di un oggetto della classe esterna, tuttavia nel caso di classe interna dichiarata static (detta anche nested) non è necessario disporre di tale oggetto.

Tornando all'esempio di contenitore generico, a questo punto è possibile creare uno Stack che contiene solo Stringhe:

≡ Esempio Stack solo stringhe

```

LinkedStack<String> names = new LinkedStack<String>();

void printNames(LinkedStack<String> names) {
    String nextName = names.pop(); // no casting needed!
    names.push("John"); // ok
    names.push(new Integer(3)); // compile-time error! This stack
    contains only String
}

```

Tramite la parametrizzazione, un tentativo di immissione di un dato incoerente nella struttura genera tempestivamente un errore a tempo di compilazione, garantendo la sicurezza (type safety) senza oneri aggiuntivi di casting in fase di estrazione.

Interfacce per Generics

Le Java Generics possono essere anche utilizzate per parametrizzare la dichiarazione di interfacce

≡ Esempio di dichiarazione di interfacce

```

public interface Generator <T> { T next(); }

```

L'interfaccia Generator garantisce che il metodo `next()` restituisca un valore del tipo specificato dal parametro `T`. Un'altra interfaccia parametrizzata è `Iterable` che forza l'implementazione del metodo:

```

Iterator<T> iterator()

```

Le classi che implementano tale interfaccia permettono ad un oggetto di essere usato nello statement "foreach".

Metodi generici

Oltre all'astrazione strutturale, il linguaggio permette la definizione di metodi generici, parametrizzando unicamente le dichiarazioni delle specifiche funzioni all'interno di una

classe.

Un metodo può essere definito generico indipendentemente dalla fatto che la classe sia generica oppure no. Per di più, se un metodo definito in una classe parametrizzata è statico, tale metodo non accederà al parametro di tipo della classe.

Per definire un metodo come generico è sufficiente parametrizzare la sua dichiarazione

≡ Esempio di metodo generico

```
public class GenericMethods {
    public <T> void f(T x) {
        System.out.println(x.getClass().getName());
    }
    public static void main(String[] args) {
        GenericMethods gm = new GenericMethods();
        gm.f("");
        gm.f(1);
        gm.f(1.0);
        gm.f(1.0F);
        gm.f('c');
        gm.f(gm);
    }
}
```

Nell'esempio, la funzione `f()` è stata “sovraccaricata” (overloaded) ben sei volte. `f()` accetterà anche valori di tipo primitivo mediante il meccanismo dell'autoboxing.

Java introduce anche una semplificazione che permette di inferire (definire) il tipo in maniera automatica

```
List <String> ls = new ArrayList();
```

I metodi generici possono essere anche utilizzati in combinazione con metodi a numero variabile di argomenti

Problema dell'erasure

Le Java Generics lasciano comunque alcune questioni poco chiare.

Per esempio, mentre è possibile ricorrere al letterale di classe per la classe `ArrayList`:

```
ArrayList.class
```

non è possibile ricorrere al letterale di classe per la classe ArrayList ottenuta parametrizzando il tipo del contenuto:

```
ArrayList<Integer>.class
```

Il compilatore Java adotta la **tecnica dell'erasure**: tutte le direttive e i controlli sui tipi vengono eseguiti ed esauriti a compile-time, per poi essere cancellati durante la generazione del bytecode. Conseguentemente, a tempo di esecuzione, le istanze di collezioni diverse dal punto di vista semantico risultano essere indistinguibili e appartenenti al medesimo tipo base grazie alla tecnica dell'astrazione generica.

Wildcards

All'interno delle Generics possono essere messi dei tipi parametrici jolly chiamati **wildcard**, che servono ad indebolire i vincoli di tipo

Wildcards

```
public class WildcardClassReferences {
    public static void main(String[] args) {
        Class<?> intClass = int.class; //? significa di qualunque tipo
        intClass = double.class;
    }
}
```

In alcuni casi un riferimento potrebbe essere troppo generico, quindi al fine di creare riferimenti specifici è possibile combinare una wildcard con una estensione di una classe

Esempio di estensione

```
public class BoundedClassReferences {
    public static void main(String[] args) {
        Class<? extends Number> bounded = int.class;
        bounded = double.class;
        bounded = Number.class;
    }
}
```

```

    }
}

```

Identificazione di tipo a run-time

Java permette di scoprire informazioni sugli oggetti e sulle classi a run-time, basandosi su due approcci:

- RTTI (Run-time Type Identification, RTTI): si presuppone che le informazioni di tutti i tipi sono accessibili sia in fase di compilazione che di esecuzione
- Meccanismo di riflessione: permette di scoprire informazioni sulle classi esclusivamente al run-time

RTTI tradizionale

La rappresentazione delle informazioni su un tipo viene realizzato tramite un tipo speciale di oggetto chiamato **Class** (talvolta chiamato meta classe).

Contiene diverse informazioni, tra cui metodi, attributi, modalità di accesso... e durante la compilazione ogni classe che costituisce un programma ha un oggetto Class

Gli oggetti di Class relativi alle varie classi che compongono un programma non sono caricati tutti in memoria prima di iniziare l'esecuzione, infatti quando in run-time si istanzia una classe, la Java Virtual Machine (JVM), su cui sta girando il programma, prima verifica se l'oggetto Class corrispondente è caricato e in caso negativo la JVM lo carica cercando il file .class con quel nome

☰ Esempio di RTTI tradizionale

```

class Candy {
    static { // questa è una clausola statica
        System.out.println("Loading Candy");
    }
}

class Cookie {
    static { // questa è una clausola statica
        System.out.println("Loading Cookie");
    }
}

public class SweetShop {
    public static void main(String[] args) {
        System.out.println("inside main");
    }
}

```

```

new Candy();
System.out.println("After creating Candy");
try {
    Class.forName("Gum");
} catch(ClassNotFoundException e) {
    e.printStackTrace(System.err);
}
System.out.println("After Class.forName(\"Gum\")");
new Cookie();
System.out.println("After creating Cookie");
}
}

```

In questo esempio, ognuna delle classi Candy e Cookie ha una clausola statica che viene eseguita quando la classe è caricata la prima volta

Il metodo `forName()` è un metodo statico di `Class` che serve per ottenere un riferimento a un oggetto `Class`, esso prende un oggetto di tipo `String` contenente il nome testuale della classe di cui si vuole il riferimento e restituisce un riferimento a `Class`. Si può notare come ogni oggetto `Class` è stato caricato solo quando era necessario.

Alternativamente, per ottenere un riferimento a un oggetto `Class` **si può anche ricorrere al letterale di classe (class literal)**, dato dal nome della classe seguito da `.class` (esempio: `Gum.class`).

I vantaggi di questa notazione sono:

- Semplicità
- Efficienza
- Controllo di esistenza della classe durante la compilazione

Il letterale di classe funziona anche con gli array, tipi primitivi (`boolean.class`) e interfacce. Per i "wrapper" dei tipi primitivi c'è anche un campo standard chiamato **TYPE**, che produce un riferimento all'oggetto `Class` per il tipo primitivo associato tale che si hanno le seguenti equivalenze

Equivalenze di tipo TYPE

... is equivalent to ...	
boolean.class	Boolean.TYPE
char.class	Character.TYPE
byte.class	Byte.TYPE
short.class	Short.TYPE
int.class	Integer.TYPE
long.class	Long.TYPE
float.class	Float.TYPE
double.class	Double.TYPE
void.class	Void.TYPE

Forme di RTTI

Le forme di RTTI viste finora sono:

- Il classico cast che usa RTTI per assicurarsi che il cast è corretto e solleva una eccezione `ClassCastException` se è stato ottenuto un cast non corretto
- L'oggetto `Class` rappresentante il tipo dell'oggetto. L'oggetto `Class` può essere interrogato per ottenere utili informazioni al run-time

In Java, che esegue il controllo di tipo, questo tipo di cast è spesso chiamato "type safe downcast"

Un'altra forma di RTTI in Java è ottenuta attraverso l'uso della parola chiave `instanceof`, che indica se un oggetto è istanza di un particolare tipo e restituisce un boolean.

L'uso dell'operatore `instanceof` potrebbe risultare spesso molto noioso perché lo si deve specificare per il confronto di ogni tipo di oggetto distinto, quindi la classe `Class` mette a disposizione il metodo `isInstance()` che fornisce un modo per invocare dinamicamente l'operatore `instanceof`

Quando si dispone di un oggetto, si può estrarre il riferimento all'oggetto `Class` relativo alla sua classe richiamando un metodo che è implementato in `Object`: `getClass`

Meccanismo di riflessione

Talvolta le informazioni sulla classe dell'oggetto non sono accessibili a tempo di compilazione, in tal caso risulta molto utile poter usufruire di un meccanismo che ricava le

informazioni relative alla classe al run-time

La classe `Class` supporta il concetto di riflessione e c'è una libreria aggiuntiva `java.lang.reflect` che contiene delle classi utili allo scopo: `Field`, `Method`, `Constructor` (ognuno dei quali implementa una interfaccia `Member`).

Questo tipo di oggetti sono creati dalla JVM al run-time per rappresentare il corrispondente membro della classe sconosciuta.

Quando si usa il meccanismo di riflessione, la JVM tratta l'oggetto come appartenente ad una classe particolare, per questo deve essere sempre accessibile (localmente e in rete)

JDBC

Java ha la possibilità di sviluppare applicazioni client/server indipendenti dalla piattaforma, garantita anche per applicazioni che lavorano su basi di dati.

Java implementa lo standard JDBC (Java DataBase Connectivity) che è **platform-independent**, e fornisce un driver per poter gestire dinamicamente tutti gli oggetti driver di cui hanno bisogno le interrogazioni a database.

JDBC incorpora in se stesso tutte le normali operazioni di interfacciamento con un database: connessione, creazione di tabelle, interrogazione e visualizzazione dei risultati

Attraverso il driver JDBC si possono effettuare tutte le operazioni disponibili su un DMBS.

Connessione ad un database

Per poter aprire una connessione ad un database è necessario ottenere un oggetto di tipo `Connection`, che fornisce tutti i metodi per preparare le query SQL.

Per ottenere una connessione è necessario caricare il driver che implementa le API JDBC, chiamando il metodo `getConnection()` della classe `DriverManager`

Il metodo `DriverManager.getConnection` stabilisce una connessione ad un database. Questo metodo richiede una database URL, che dipende dal DBMS, per esempio:

≡ Esempio di connessione database H2

```
jdbc:h2:/home/user/test/db
```

dove `/home/user/test/db` è il file su file system che conterrà il DB

Altri parametri come ad esempio username e password possono essere specificati attraverso un oggetto `Properties` passato al metodo `getConnection()` insieme alla URL

☰ Esempi di connessione al database

```
//connessione senza parametri
Connection conn =
DriverManager.getConnection("jdbc:h2:/home/user/test/db");
//connessione con username e password
Connection conn =
DriverManager.getConnection("jdbc:h2:/home/user/test/db","us
er","1234"); //connessione con oggetto Properties
Properties dbprops = new Properties();
dbprops.setProperty("user", "user");
dbprops.setProperty("password", "1234");
Connection conn =
DriverManager.getConnection("jdbc:h2:/home/user/test/db", dbprops);
```

SQLException

Quando la JDBC genera un errore durante le interrogazioni di un database solleva un'eccezione di tipo **SQLException**, che contiene diverse informazioni:

- Una descrizione testuale, data dal metodo `getMessage()`
- `getSQLState()`, restituisce un codice alfanumerico codificato secondo lo standard ISO/ANSI e Open Group (X/Open)
- `getErrorCode()` restituisce un valore intero che indica un codice di errore specifico del driver che implementa JDBC

Statement

Le query SQL si eseguono attraverso oggetti di tipo `Statement`, ottenuti tramite l'oggetto `Connection`.

È possibile ottenere anche degli statement preimpostati in cui è possibile sostituire a dei segnaposto inseriti nella query SQL dei valori. Queste query preimpostate sono utili per inserire in maniera corretta all'interno della query dei letterali applicando le opportune conversioni di tipo

Statement di modifica

Per eseguire le varie operazioni di modifica di un DB (come creazione di tabelle, aggiunta di tuple, modifica di tuple) si usa il metodo `executeUpdate()` definito dall'oggetto `Statement`. Gli statement vanno sempre chiusi tramite `close()` per liberare risorse

≡ Esempio di statement di modifica

```
public static final String CREATE_TABLE = "CREATE TABLE IF NOT EXISTS
store (artId INT PRIMARY KEY, desc VARCHAR(1024), price DOUBLE, unit
INTEGER)";

...

Connection conn =
DriverManager.getConnection("jdbc:h2:/home/user/db/store", dbprops);
Statement stm = conn.createStatement();
stm.executeUpdate(CREATE_TABLE);
stm.close(); //chiudere lo statement!!!

stm = conn.createStatement();
stm.executeUpdate("INSERT INTO store VALUES(1, 'pentola', 4.5, 20)");
stm.close();
```

≡ Esempio di prepared Statement

```
PreparedStatement pstmt = conn.prepareStatement("INSERT INTO store
VALUES (?, ?, ?, ?)"); // ? è un segnaposto
pstmt.setInt(1, 2); //l'indice parte da 1
pstmt.setString(2, "piatto"); // i metodi set si occupano di inserire i
letterali nella query SQL
pstmt.setDouble(3, 1.5); pstmt.setInt(4, 40);
pstmt.executeUpdate();
pstmt.close();
```

Statement di interrogazione

Le query di selezione dei dati si effettuano con il metodo `executeQuery("Query")` che è istanziato sempre da `Statement`.

`executeQuery()` restituisce un oggetto di tipo `ResultSet` che permette di navigare nelle tuple restituite (simile ad un iteratore)

≡ Esempio di statement di interrogazione

```
Statement stm = conn.createStatement();
ResultSet rs = stm.executeQuery("SELECT artId, desc FROM store WHERE
unit>5");
while (rs.next()) {
    System.out.println(rs.getInt(1) + ": " + rs.getString(2));
}
rs.close();
stm.close();
```

```
PreparedStatement pstmt = conn.prepareStatement("SELECT artId, desc
FROM store WHERE unit > ?");
pstmt.setInt(1, 20);
rs = pstmt.executeQuery();
while (rs.next()) {
    System.out.println(rs.getInt(1) + ": " + rs.getString(2));
}
rs.close();
stm.close();
```

È possibile utilizzare anche i PreparedStatement per le SELECT

Programmazione concorrente

Un calcolatore moderno è progettato per eseguire molteplici compiti simultaneamente, come la riproduzione musicale, la navigazione sul web o la stesura di un documento in parallelo.

Java mette a disposizione dei programmatori una vasta gamma di classi dedicate alla programmazione concorrente, le quali sono principalmente racchiuse nel pacchetto standard

```
java.util.concurrent
```

Processi e thread

Anche in presenza di un calcolatore dotato di una singola CPU, il sistema è in grado di simulare l'esecuzione simultanea di più processi attraverso una tecnica denominata **time slicing**, ossia il suddividere il tempo di calcolo complessivo della CPU in finestre temporali, distribuendolo tra i vari processi attivi.

I **processi** rappresentano tipicamente una singola applicazione in esecuzione all'interno del sistema operativo e possono essere composti da **threads**, ossia delle unità di esecuzione meno complesse dei processi.

Runnable e Thread

In Java, le funzionalità inerenti alla gestione e al ciclo di vita di un singolo flusso di esecuzione sono implementate e fornite dalla classe `Thread`.

La creazione di un thread operativo può avvenire seguendo due approcci:

- **Approccio per classe da zero**
- **Approccio per estensione di classe**

≡ Esempio di approccio per classe da zero

Si va a creare una classe che implementa l'interfaccia `Runnable`: questa interfaccia prevede un singolo metodo `run()` dove va inserito il codice che deve eseguire il thread

```
public class HelloRunnable implements Runnable {
    public void run() {
        System.out.println("Hello from a thread!");
    }

    public static void main(String args[]) {
        (new Thread(new HelloRunnable())).start();
    }
}
```

≡ Esempio di approccio per estensione di classe

```
public class HelloThread extends Thread {
    public void run() {
        System.out.println("Hello from a thread!");
    }

    public static void main(String args[]) {
        (new HelloThread()).start();
    }
}
```

In entrambi i casi è necessario invocare il metodo `start()` di `Thread` per farlo partire. L'interfaccia `Runnable` è più generica in quanto svincolata dalla classe `Thread`, il suo

utilizzo permette di evitare l'ereditarietà delle varie altre classi da `Thread`, così da poter ereditare da altre se necessario

Controllo di esecuzione

Sospensione di esecuzione

Per regolare il ritmo di esecuzione è possibile sospendere il thread corrente invocando il metodo `Thread.sleep()`.

L'intervallo di tempo può essere espresso in millisecondi o millisecondi+nanosecondi, anche se il calcolo del tempo è dipendente dal sistema operativo e non può essere esatto.

≡ Esempio di sleep

```
public class SleepMessages {
    public static void main(String args[]) throws InterruptedException
    { // sleep può generare un'eccezione se il thread viene interrotto da
      un altro thread durante lo sleep
        String importantInfo[] = {
            "Info 1",
            "Info 2",
            "Info 3",
            "Info 4"
        };
        for (int i = 0; i < importantInfo.length; i++) {
            //sospendi per 4 secondi (4000 millisecondi)
            Thread.sleep(4000);
            //Stampa il messaggio
            System.out.println(importantInfo[i]);
        }
    }
}
```

Interruzione di un thread

L'interruzione di un thread in esecuzione può essere forzata invocando il metodo `interrupt()`.

Ogni thread, per essere ben implementato, deve avere le seguenti caratteristiche:

- Ogni thread deve implementare il suo metodo `interrupt()`
- Ogni `interrupt()` deve coincidere con la sua terminazione

- Ogni thread cattura l'eccezione `InterruptedException` e interrompa la sua esecuzione

≡ Esempi di interrupt

Riprendendo l'esempio precedente, catturiamo l'eccezione durante lo sleep

```
for (int i = 0; i < importantInfo.length; i++) {
    // Pause for 4 seconds
    try {
        Thread.sleep(4000);
    } catch (InterruptedException e) {
        // We've been interrupted: no more messages.
        return; // Se interrotto esco dal metodo run
    }
    // Print a message
    System.out.println(importantInfo[i]);
}
```

Se non ci sono metodi che generano `InterruptedException` allora controlliamo periodicamente se qualche altro thread abbia invocato l'interruzione

```
for (int i = 0; i < inputs.length; i++) { //do something
    if (Thread.interrupted()) { //Restituisce true nel caso di
        interruzione
        // We've been interrupted return;
    }
}
```

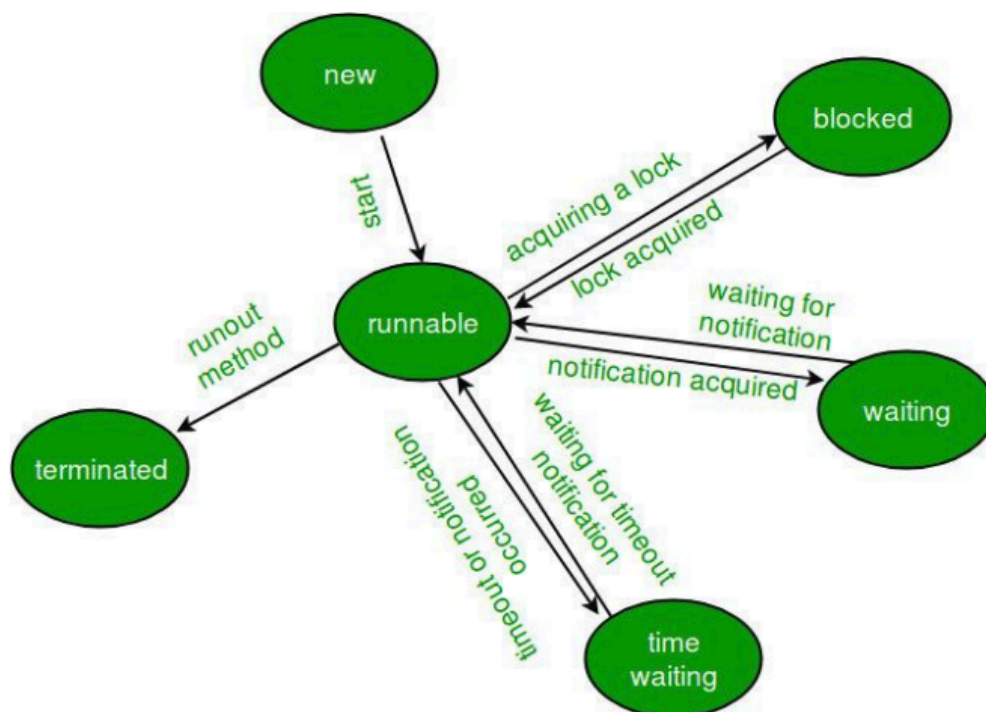
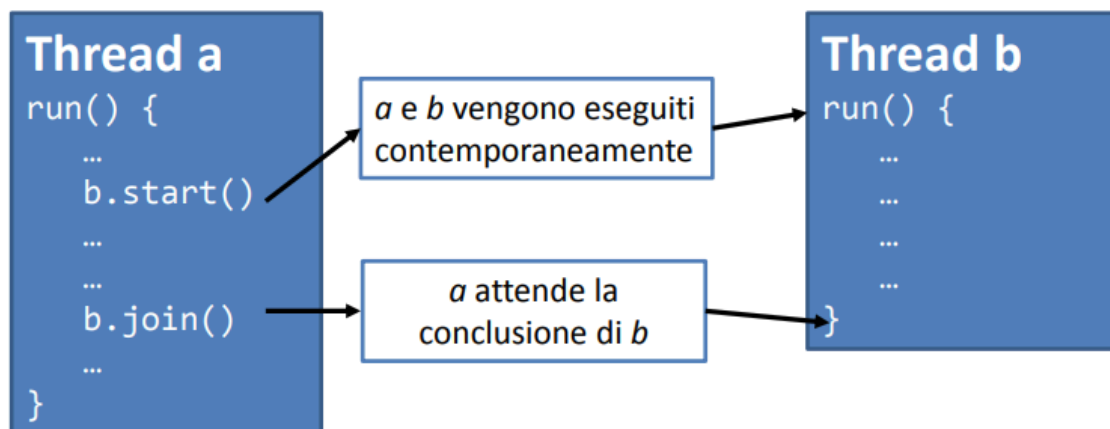
Unione di esecuzione dei thread

La classe `Thread` include anche il metodo `join()`, utile per la temporizzazione dei stessi thread

La chiamata del metodo `join()` mette in pausa il thread corrente fino a quando il thread sul quale si è chiamato non termina. Come il metodo precedente, `join()` permette di specificare un tempo massimo di atteso e può essere interrotto da un

`InterruptedException`

📘 Visualizzazione grafica del join e del ciclo di vita di un thread



Sincronizzazione dei thread

I thread per poter comunicare e accedere alle stesse risorse hanno bisogno di essere **sincronizzati**.

Quando due thread non sono sincronizzati incorrono due problematiche:

- **Thread interference**
- **Memory consistency error**

La sincronizzazione permette di risolverle ma introduce problemi di **thread contention** quando vogliono utilizzare le stesse

Un metodo sincronizzato può essere chiamato da un solo thread alla volta (e gli altri restano in attesa), garantendo che il metodo protetto può essere invocato ed eseguito soltanto da un

thread per volta, forzando tutti gli eventuali altri thread richiedenti a rimanere in uno stato di attesa

☰ Esempio di metodi sincronizzati

```
public class SynchronizedCounter {
    private int c = 0;
    public synchronized void increment() {
        c++;
    }
    public synchronized void decrement() {
        c--;
    }
    public synchronized int value() {
        return c;
    }
}
```

Thread interference

Una **thread interference** avviene quando due operazioni su due thread differenti agiscono sullo stesso dato.

Essendo operazioni non atomiche è difficile ottenere un risultato prevedibile

☰ Esempio di thread interference

```
class Counter {
    private int c = 0;
    public void increment() {
        c++;
    }
    public void decrement() {
        c--;
    }
    public int value() {
        return c;
    }
}
```

L'operazione `c++` (`c--`) non è atomica, poiché

1. Recupera il valore di `c`
2. Incrementa (o decrementa) il valore assegnato
3. Assegna a `c` il nuovo valore

Supponiamo che ci siano due thread, `A` e `B`, il primo richiami la funzione `increment` e il secondo `decrement`:

1. Thread A: Retrieve `c`
 2. Thread B: Retrieve `c`
 3. Thread A: Increment retrieved value; result is 1
 4. Thread B: Decrement retrieved value; result is -1
 5. Thread A: Store result in `c`; `c` is now 1
 6. Thread B: Store result in `c`; `c` is now -1
- `c` sarà uguale a -1 e l'operazione di A viene persa

Memory inconsistency

Si verifica quando due thread hanno una visione inconsistente di uno stesso dato.

Si considera un tipico scenario in cui una variabile inizializzata al valore 0 sia accessibile simultaneamente a due entità, definite come Thread `A` e Thread `B`. Qualora il Thread `A` procede all'incremento della variabile e, nel medesimo istante temporale, il Thread `B` proceda alla stampa a schermo della stessa, sussiste la concreta possibilità che il Thread `B` restituisca il valore originario di 0 poiché l'operazione di aggiornamento del Thread `A`, benché avviata, non si è ancora conclusa.

Sincronizzazioni delle istruzioni

È possibile anche sincronizzare soltanto una porzione di istruzioni, utile quando si richiede che soltanto quella porzione venga "velocizzata"

☰ Esempio di sincronizzazione di funzione

```
public void addName(String name) {
    synchronized(this) { //in questo caso sincronizziamo solo la
        modifica di lastName e di nameCount. L'istruzione add non richiede
        sincronizzazione. Stiamo inserendo un lock sull'oggetto
        lastName = name;
        nameCount++;
    }
}
```

```
nameList.add(name);
}
```

Problemi della programmazione concorrente

Ci sono diversi problemi presenti nella programmazione concorrente:

- **Deadlock**: due o più thread sono bloccati in modo indefinito perché ognuno attende la fine dell'altro
- **Starvation**: un thread non riesce ad accedere ad alcune risorse perché sono utilizzate avidamente da altri thread
- **Livelock**: un thread **A** in genere agisce in risposta di un altro thread **B**, se **B** agisce in risposta di un altro thread **C** allora i thread proseguiranno in maniera discontinua

Programmazione in rete

La programmazione in rete, nota anche come programmazione distribuita, è sempre risultata complessa e particolarmente soggetta ad errori data dalla necessità da parte del programmatore di conoscere tutto il

Nel linguaggio Java, la programmazione in rete subisce una notevole semplificazione, venendo astratta in modo efficace attraverso l'uso di un set dedicato di classi.

Queste classi gestiscono dei file di lettura e scrittura che non si trovano nella macchina locale ma in una remota, con totale autonomia di come processare un'informazione inviata o richiesta.

Il modello di programmazione adottato si fonda sull'incapsulamento (**wrapping**) di una connessione di rete, definita socket, all'interno di un flusso (**stream**) di oggetti. Grazie a questa astrazione è possibile impiegare le medesime invocazioni di metodo utilizzate per gli stream dei file.

Grazie alla natura multiplatform di Java, tutte le specificità e i dettagli di basso livello della rete vengono gestiti direttamente dalla Java Virtual Machine (JVM) e dall'installazione locale dell'ambiente Java

Identificazione di una macchina

Per instaurare una comunicazione efficace con un altro nodo della rete, risulta indispensabile identificare in maniera univoca il destinatario.

L'identificazione del nodo avviene mediante l'indirizzo IP (Internet Protocol).

Esistono fondamentalmente due approcci per risolvere e identificare un indirizzo IP:

- L'utilizzo del DNS (Domain Name System), ricorrendo a stringhe alfanumeriche come ad esempio *www.di.uniba.it*

- L'impiego della dot notation, che prevede l'uso di sequenze numeriche come 183.201.181.10.

In Java, la rappresentazione dell'indirizzo IP, in entrambe le notazioni appena descritte, è affidata a una classe specifica denominata `InetAddress`, la quale è inclusa nel package `java.net`.

Questa classe mette a disposizione del programmatore il metodo statico

`InetAddress.getByName()`, il cui scopo è restituire un'istanza di `InetAddress` partendo dal nome dell'host o dal suo indirizzo IP.

Esempio di socket

```
//null sta per localhost
InetAddress add = InetAddress.getByName(null);
System.out.println(add);
add = InetAddress.getByName("localhost");
System.out.println(add);
add = InetAddress.getByName("www.google.it");
System.out.println(add);
```

Uso del port

Su una macchina singola può ospitare più servizi contemporaneamente, per questo l'indirizzo IP da solo non è sufficiente.

Quando si imposta un client o un server è necessario scegliere la “porta” (port) sul quale sia il server che il client decidono di connettersi.

Il port non è una locazione fisica su una macchina ma è una astrazione software, tipicamente ogni servizio è associato ad un singolo numero di port su una macchina server. Il programma clienti quindi non deve conoscere soltanto l'indirizzo IP, ma anche la porta giusta.

Socket

All'interno dell'ecosistema Java, la connessione verso una macchina remota viene stabilita attraverso l'utilizzo dei **socket**. Il socket è un astrazione software usata per rappresentare i terminali di connessioni di due macchine

Le librerie di Java mettono a disposizione due classi principali basate sugli stream per la gestione dei socket:

- La classe `ServerSocket`, impiegata dal server per rimanere in ascolto delle richieste di connessione in ingresso
- La classe `Socket`, utilizzata dal client per inizializzare attivamente la connessione

Creando un socket in Java, si ottengono un `InputStream` e un `OutputStream` (o, con appropriate conversioni, un `Reader` e un `Writer`) al fine di abilitare la connessione in modo simile a un I/O su stream di oggetti.

Una volta che un client richiede una connessione socket, il `ServerSocket` restituisce (mediante il metodo `accept()`) un `Socket` corrispondente attraverso il quale la comunicazione può avvenire dal lato server, creando una connessione **Socket-To-Socket**

Durante la fase di inizializzazione, la creazione di un `ServerSocket` richiede esclusivamente l'indicazione di un numero di porta, omettendo l'indirizzo IP poiché esso coincide implicitamente con quello della macchina su cui il server è in esecuzione. Al contrario, la creazione di un `Socket` lato client impone la specificazione di entrambi i parametri, in quanto server e client risiedono generalmente su elaboratori distinti.

Il socket generato dalla chiamata `ServerSocket.accept()` incapsulerà automaticamente sia le informazioni del client sia quelle del server.

Raggiunto questo punto, si possono invocare i metodi `getInputStream()` e `getOutputStream()` sui rispettivi socket per ricavare gli stream di dati, i quali supportano nativamente l'impiego delle classi di buffering e di formattazione del testo.

Servire più client

Per consentire al server di servire più client in maniera simultanea, è indispensabile ricorrere al multithreading.

Il design pattern di base per affrontare tale casistica prevede l'istanziamento di un singolo `ServerSocket` sul server, seguito dalla chiamata bloccante al metodo `accept()`, che pone il processo in attesa attiva di una connessione.

Nel momento in cui una connessione viene stabilita e il metodo `accept()` conclude la sua esecuzione restituendo il socket di comunicazione, quest'ultimo viene immediatamente passato a un nuovo thread appositamente istanziato per servire le richieste di quello specifico client. Nel frattempo, il thread principale del server non si arresta, ma si riavvia in un ciclo perpetuo richiamando nuovamente il metodo `accept()`, mettendosi così in attesa della successiva richiesta di connessione.

Esempi di codice

Poiché gli esempi di codice sono molto prolissi, rimando direttamente al materiale fornito dal professore sulla sua [pagina Github](#)

Java RESTful

Il REST (**Representational State Transfer**) è un tipo di architettura software per i sistemi distribuiti, basato sulla trasmissione di dati tramite protocollo HTTP senza ulteriori livelli.

I sistemi REST non prevedono il concetto di sessione (anche chiamati **stateless**), ma prevedono una struttura URI (Uniform Resource Identifier) ben definita, che identifica univocamente una risorsa o un insieme di risorse.

Principi di una REST

Nella REST, lo stato dell'applicazione e le funzionalità sono divisi in **risorse web**, ognuna di esse unica e indirizzabile usando la sintassi delle URI.

Comunemente viene definita un'interfaccia uniforme per la condivisione delle risorse che permette il trasferimento di stato tra client e risorse, attraverso:

- Un insieme vincolato di operazioni ben definite
- Un insieme vincolato di contenuti
- Un protocollo che è:
 - **Client-server**: I ruoli del client e del server sono ben separati, diventa più facile poter scalare con il tempo e server e client possono essere sostituiti e sviluppati indipendentemente fintanto che l'interfaccia non viene modificata.
 - **Privo di stato (stateless)**: La comunicazione client-server è vincolata in modo che nessun contesto client venga memorizzato sul server tra le richieste. Ciascuna richiesta dai vari client contiene tutte le informazioni necessarie per richiedere il servizio e lo stato della sessione è contenuto nel client
 - **Memorizzabile in cache**: I client possono mettere in cache le risposte. Queste devono in ogni modo definirsi implicitamente o esplicitamente cacheable o no, in modo da prevenire che i client possano riutilizzare stati vecchi e dati errati.
 - **A livelli**: il sistema è realizzato "a strati" (layer). Ciò rende possibile, per esempio, pubblicare le API in un server, memorizzare i dati in un secondo server e gestire l'autenticazione delle richieste in un terzo server.

Risorse

Una risorsa è una fonte di informazioni alla quale si può accedere attraverso un identificativo univoco (URI).

Per utilizzare tali risorse sia il client e il server comunicano attraverso un **protocollo comune** (HTTP) e che sia il server che il client abbiano un **meccanismo di rappresentazione** (come JSON o XML).

RESTful contrariamente ad altri protocolli non definisce standard per la rappresentazione o per le modalità di comunicazione.

Rappresentazione dei dati

Un formato molto utilizzato per lo scambio dei dati è **JavaScript Object Notation (JSON)**.

Un JSON è racchiuso tra parentesi graffe e può contenere diversi tipi di dati.

☰ Esempio di rappresentazione JSON

```
{
  "name": "Mario",
  "surname": "Rossi",
  "active": true,
  "favoriteNumber": 42,
  "birthday": {
    "day": 1,
    "month": 1,
    "year": 2000
  },
  "languages": [ "it",
"en" ]
}
```

Definizione dei metodi

Progettare una API REST prevede la definizione dei **metodi/servizi**.

Ogni metodo ha un suo **path**, ossia un identificativo univoco che coincide con la URL da utilizzare per richiedere il metodo tramite protocollo HTTP. Ad ogni metodo deve essere associato anche il relativo verbo HTTP da utilizzare.

Le URL possono avere dei parametri che possono essere utilizzati per scambiare informazioni con il server, ad esempio l'id della risorsa da recuperare.

☰ Esempio di risorsa da recuperare tramite URL

Supponiamo di avere un metodo per il recupero di un libro da un store:

```
http://store.api.org/book?id=12
```

```
http://store.api.org/book/12
```

Riprendendo l'esempio precedente il client potrebbe utilizzare uno dei due servizi semplicemente effettuando una richiesta HTTP/GET per ottenere un libro tramite il suo id (es. 12):

```
{
  "id": 12,
  "title": "Alice in Wonderland",
}
```

```
"price": 12.50
}
```

Java e JSON

Per poter integrare JSON con il linguaggio Java utilizzeremo una libreria di nome **GSON**

≡ Esempio di integrazione con il libro

Gson



```

public class Book {
    private String title;
    private String isbn;
    private String[] authors;
    private double price;

    public Book(String title, String isbn, String[] authors, double price) {
        this.title = title;
        this.isbn = isbn;
        this.authors = authors;
        this.price = price;
    }
}

{
    "title": "Lo Zen e l'arte della manutenzione della motocicletta",
    "isbn": "9788845902826",
    "authors": ["Robert M. Pirsig"],
    "price": 12.5
}



```

 @param args the command line arguments
 */
 public static void main(String[] args) {
 Book book = new Book("Lo Zen e l'arte della manutenzione della motocicletta",
 "9788845902826",
 new String[]{"Robert M. Pirsig"},
 12.50);
 Gson gson = new Gson();
 //Object -> JSON
 String jsonString = gson.toJson(book);
 System.out.println(jsonString);
 //JSON -> Object
 Book book2 = gson.fromJson(jsonString, Book.class);
 System.out.println(book2);
 }

```


```

Possiamo vedere come la classe GSON implementi i metodi `toJson()` per convertire i parametri passati in un file JSON a partire da un oggetto e come possa fare la conversione inversa con `fromJson()`

JAX-RS

Jakarta RESTful Web Services, (JAX-RS) è una API di Java che fornisce supporto nella creazione di servizi Web secondo il modello architetturale REST.

JAX-RS utilizza le **annotazioni**, per semplificare lo sviluppo e la distribuzione di client e endpoint di servizi Web.

≡ Esempio di annotazione



```

@Path("book")
public class BookService {

    @GET
    @Produces("application/json")
    public Response getBookById(@DefaultValue("-1") @QueryParam("id") String bookid) {
        //CODE HERE
    }

    @GET
    @Path("/{bookid}")
    @Produces("application/json")
    public Response book(@PathParam("bookid") String bookid) {
        //CODE HERE
    }
}

```

esempio di annotazione, iniziano con il carattere '@'

Annotazioni

Esistono varie annotazioni all'interno di JAX-RS

- **@Path**: specifica il path all'interno della URL dove sarà disponibile il servizio la cui logica è realizzata all'interno di un metodo. Il path può essere sia a livello di classe sia a livello di metodo
- **@Produces/@Consumes**: specifica il data type che viene prodotto e/o consumato
- **@GET, @POST, @DELETE...**: specificano il verbo HTTP con il quale sarà richiamato il relativo servizio
- **@QueryParam**: identifica un parametro che verrà passato tramite parametro della URL
- **@PathParam**: identifica un parametro che verrà passato tramite path della URL

☰ Esempio di annotazioni per metodo

```

1
@Path("book")
public class BookService {

    2
    @GET
    @Produces("application/json")
    public Response getBookById(@DefaultValue("-1") @QueryParam("id") String bookid) {
        //CODE HERE
    }

    3
    @GET
    @Path("/{bookid}")
    @Produces("application/json")
    public Response book(@PathParam("bookid") String bookid) {
        //CODE HERE
    }
}

```

1. definisce il "root" path associato a tutti i metodi presenti nella classe
2. definisce un metodo GET e il tipo di dati prodotto, inoltre viene definito un parametro di tipo query con valore di default =-1
3. viene definito un path aggiuntivo per questo metodo e un parametro (*bookid*) definito all'interno del path

Esempio di annotazione per un metodo PUT:

```

@PUT
@Path("/add")
@Consumes("application/json")
@Produces("application/json")
public Response add(String json) {
    //CODE HERE
}

```

In questo caso il parametro `json` conterrà il contenuto della richiesta HTTP passato tramite PUT.

☰ Esempio libri

Implementiamo la logica dei metodi per recuperare e salvare un book.

```

@Path("book")
public class BookService {

    @GET
    @Produces("application/json")
    public Response getBookById(@DefaultValue("-1") @QueryParam("id") String bookid) {
        System.out.println("Book id: " + bookid);
        Book book = new Book("Lo Zen e l'arte della manutenzione della motocicletta",
                               "9788845902826",
                               new String[]{"Robert M. Pirsig"},
                               12.50);
        Gson gson = new Gson();
        //Object -> JSON
        String jsonString = gson.toJson(book);
        return Response.ok(jsonString, MediaType.APPLICATION_JSON).build();
    }
}

```

Questo è il servizio GET che ci permette di recuperare un Book tramite id come parametro di query della URL. In questo caso viene restituito sempre lo stesso book, nella realtà il book potrebbe essere recuperato da un DB o altra sorgente.

Logica dei metodi

```

@GET
@Path("/{bookid}")
@Produces("application/json")
public Response book(@PathParam("bookid") String bookid) {
    System.out.println("Book id: " + bookid);
    Book book = new Book("Lo Zen e l'arte della manutenzione della motocicletta",
                           "9788845902826",
                           new String[]{"Robert M. Pirsig"},
                           12.50);
    Gson gson = new Gson();
    //Object -> JSON
    String jsonString = gson.toJson(book);
    return Response.ok(jsonString, MediaType.APPLICATION_JSON).build();
}

```

Questo è il servizio GET che ci permette di recuperare un Book tramite id come path della URL.

Da notare l'utilizzo della classe Response per costruire la risposta HTTP da restituire a seguito della chiamata.

```

@PUT
@Path("/add")
@Consumes("application/json")
@Produces("application/json")
public Response add(String json) {
    Gson gson = new Gson();
    Book book = gson.fromJson(json, Book.class);
    System.out.println("Add book: " + book);
    return Response.ok().build();
}

```

Questo è il servizio PUT che ci permette di aggiungere un Book.

La PUT è un verbo HTTP il cui contenuto, in questo caso il JSON del book da aggiungere, è inserito direttamente nella richiesta HTTP. In questo caso il parametro json memorizza il contenuto della richiesta HTTP che viene convertito in oggetto utilizzando Gson.

Server HTTP

Dopo aver creato le classi che forniscono i servizi con le relative annotazioni è necessario avere a disposizione un server HTTP che si occupi di gestire le richieste dai vari client.

Per semplicità utilizzeremo [grizzly](#), che è un server HTTP già incluso in Jersey, un framework REST per Java che permette di realizzare sia client sia server REST.

☰ Esempio di server HTTP con grizzly

```

public class RESTServer {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        URI baseUri = UriBuilder.fromUri("http://localhost/").port(4321).build();
        ResourceConfig config = new ResourceConfig(BookService.class);
        HttpServer server = GrizzlyHttpServerFactory.createHttpServer(baseUri, config);
        try {
            server.start();
            System.out.println(String.format("Jersey app started with WADL available at "
                + "%sapplication.wadl\nHit enter to stop it...", "http://localhost:4321/"));
            System.in.read();
            server.shutdown();
        } catch (IOException ex) {
            Logger.getLogger(RESTServer.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
}

```

URI che sarà la root di tutti i servizi messi a disposizione da questo server. In questo caso: <http://localhost:4321/>

```

public class RESTServer {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        URI baseUri = UriBuilder.fromUri("http://localhost/").port(4321).build();
        ResourceConfig config = new ResourceConfig(BookService.class);
        HttpServer server = GrizzlyHttpServerFactory.createHttpServer(baseUri, config);
        try {
            server.start();
            System.out.println(String.format("Jersey app started with WADL available at "
                + "%sapplication.wadl\nHit enter to stop it...", "http://localhost:4321/"));
            System.in.read();
            server.shutdown();
        } catch (IOException ex) {
            Logger.getLogger(RESTServer.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
}

```

Configura le risorse disponibili che verranno **automaticamente** create utilizzando le **annotazioni** presenti nella classe **BookService**

```

public class RESTServer {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        URI baseUri = UriBuilder.fromUri("http://localhost/").port(4321).build();
        ResourceConfig config = new ResourceConfig(BookService.class);
        HttpServer server = GrizzlyHttpServerFactory.createHttpServer(baseUri, config);
        try {
            server.start();
            System.out.println(String.format("Jersey app started with WADL available at "
                + "%sapplication.wadl\nHit enter to stop it...", "http://localhost:4321/"));
            System.in.read();
            server.shutdown();
        } catch (IOException ex) {
            Logger.getLogger(RESTServer.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
}

```

Crea il server utilizzando la URI e i servizi presenti in config e avvia il server.

```

public class RESTServer {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        URI baseUri = UriBuilder.fromUri("http://localhost/").port(4321).build();
        ResourceConfig config = new ResourceConfig(BookService.class);
        HttpServer server = GrizzlyHttpServerFactory.createHttpServer(baseUri, config);
        try {
            server.start();
            System.out.println(String.format("Jersey app started with WADL available at "
                + "%sapplication.wadl\nHit enter to stop it...", "http://localhost:4321/"));
            System.in.read();
            server.shutdown();
        } catch (IOException ex) {
            Logger.getLogger(RESTServer.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
}

```

Il server è attivo e può accettare le richieste HTTP relative ai servizi messi a disposizione. Il main rimane in esecuzione fino alla pressione del tasto INVIO.

Client REST

Dopo aver realizzato i nostri servizi REST e reso disponibile il server HTTP siamo pronti ad accettare le richieste ricevute attraverso il protocollo HTTP

Un client non deve far altro che preparare la richiesta HTTP in base alle specifiche del servizio e inoltrarla utilizzando il protocollo HTTP e il corrispettivo verbo previsto dal servizio. Java mette a disposizione nel suo framework delle classi per gestire le richieste HTTP

≡ Esempio di client con Jersey

Utilizzando il framework Jersey è semplice inviare delle richieste attraverso l'utilizzo delle classi Client e WebTarget.

```

public static void main(String[] args) {
    Client client = ClientBuilder.newClient();
    WebTarget target = client.target("http://localhost:4321");
    Response resp = target.path("book").queryParam("id", "1").request(MediaType.APPLICATION_JSON).get();
    System.out.println(resp);
    System.out.println(resp.readEntity(String.class));

    resp = target.path("book/1").request(MediaType.APPLICATION_JSON).get();
    System.out.println(resp);
    System.out.println(resp.readEntity(String.class));
}

```

Il client permette di effettuare le connessioni HTTP in modo semplice.

Il WebTarget memorizza la URL dove sono disponibili i servizi e attraverso i metodi path, queryParam, request permette di costruire la richiesta HTTP.

L'oggetto di tipo Response contiene il risultato della risposta HTTP.

☰ Esempio di richiesta PUT con invio di un JSON

```

Gson gson = new Gson();
Book book = new Book("Lo Zen e l'arte della manutenzione della motocicletta",
    "9788845902826",
    new String[]{"Robert M. Pirsig"},
    12.50);
resp=target.path("book/add").request(MediaType.APPLICATION_JSON)
    .put(Entity.entity(gson.toJson(book), MediaType.APPLICATION_JSON));
System.out.println(resp);

```

In questo caso il metodo put prende in input una entity che è il contenuto da inviare al server tramite la richiesta PUT.

SWING

SWING è il framework di Java che permette la realizzazione di interfacce grafiche (chiamate anche GUI).

Si utilizza con i packages di riferimento `javax.swing`, `javax.swing.event`.

Questo pacchetto permette di creare:

- Finestre, Form, Dialog
- Menu, Pulsanti, Check-box, Combo-box
- Alberi, Tabelle
- Layout, Look&Feel

📘 La mia prima applicazione (Netbeans+Swing)

NetBeans mette a disposizione degli strumenti che facilitano la creazione delle GUI, che introducono:

- Drag and drop dei componenti

- Auto-generazione del codice
- Strumenti per la gestione del layout dei componenti

Per trovare degli esempi conviene visionare il file [3.1 - \(Slide Esempi\) Prima Applicazione in Java SWING.pdf](#)

Contenitori

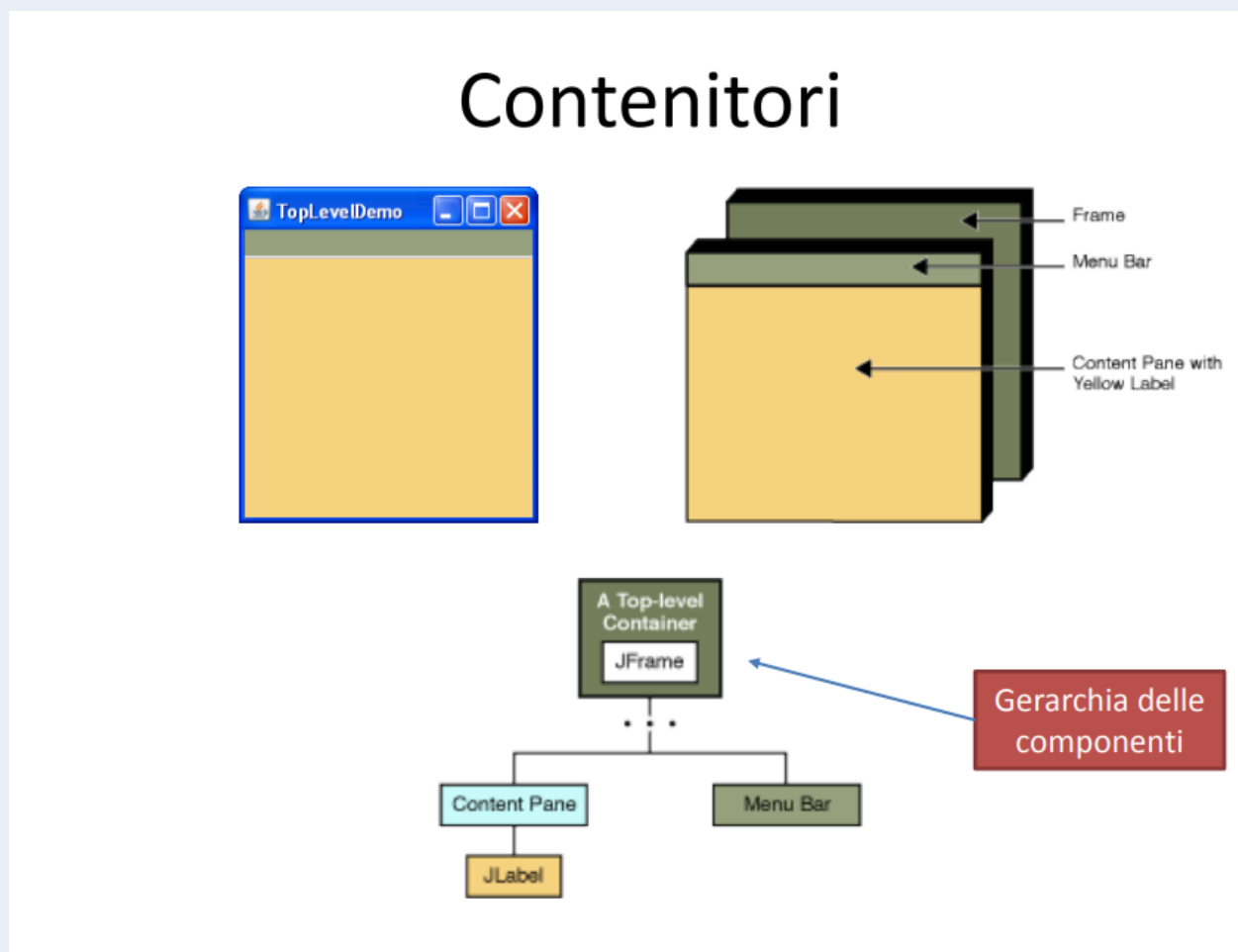
Java Swing mette a disposizione tre tipi di contenitori:

- JFrame
- JDialog
- JApplet

Ogni componente deve far parte di una gerarchia di componenti connessi ad un contenitore radice (chiamato anche **top level**), ognuno di questi componenti poi può appartenere ad un solo contenitore.

Ogni contenitore top-level è associato ad una vista (ovvero gli oggetti effettivamente visibili su schermo).

Gerarchia dei componenti



Ogni programma SWING deve avere almeno un contenitore top-level (questo

contenitore è la radice della gerarchia delle componenti)
 Per ogni contenitore top level è possibile aggiungere un menu.

Per aggiungere componenti ad un contenitore è necessario usare il metodo `add()`

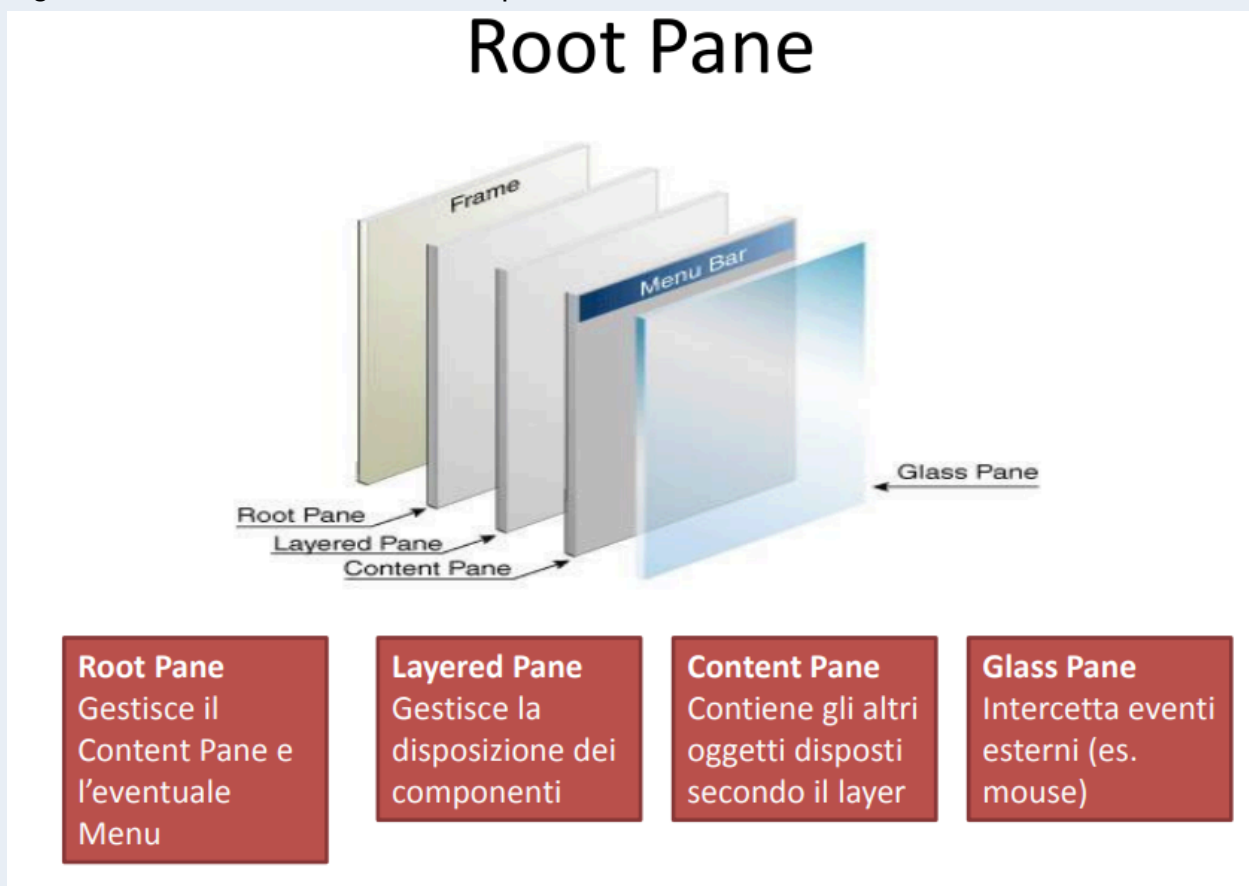
☰ Esempio di metodo `add()`

```
frame.getContentPane().add(yellowLabel, BorderLayout.CENTER)
```

`getContentPane()` restituisce un oggetto `JComponent`

📘 Il root Pane

Ogni finestra è suddivisa in diversi pannelli



JComponent

`JComponent` è una classe che mette a disposizione un set di metodi per modificare aspetto e comportamento dei componenti.

Questa eredita da `Container` che a sua volta eredita da `Component`.

`JComponent` in particolare implementa:

- Modifica dell'aspetto del componente (colore, bordo, tipo cursore)
- Monitoraggio dello stato (gestione di un PopupMenu, cambio nome, non visibile/visibile, abilita/disabilita, modifica del ToolTip)
- Gestione degli eventi
- Disegno degli oggetti
- Gestione della gerarchia degli oggetti (aggiungi, rimuovi)
- Modifica della disposizione degli oggetti (Layout)
- Dimensione e posizione del componente

Buttons, CheckBoxes, RadioButtons

I pulsanti in Swing derivano tutti dalla classe astratta `AbstractButton`. Tra le implementazioni principali troviamo:

- **JButton**: un classico pulsante
- **JCheckBox**: una check box
- **JRadioButton**: un singolo pulsante di un gruppo di pulsanti di opzione
- **JMenuItem**: una voce di un menu
- **JCheckBoxMenuItem**: una voce di un menu che è una check box
- **JRadioButtonMenuItem**: una voce di un menu che è un pulsante di opzione
- **JToggleButton**: permette di creare un bottone a due stati utilizzando due check box o due radio button

Color Chooser

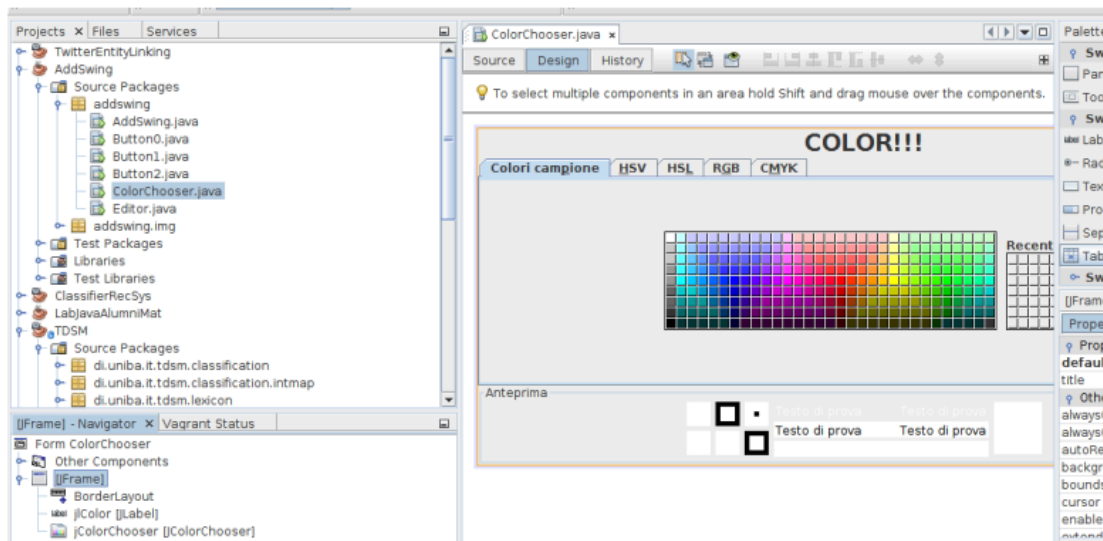
La selezione di un colore è implementata attraverso la classe `JColorChooser`, la quale offre diverse modalità di interazione per l'utente finale

Le informazioni relative al colore selezionato vengono memorizzate all'interno di un modello denominato `ColorSelectionModel()`; ogni istanza di `JColorChooser()` possiede una propria copia isolata di questo modello.

Per intercettare e gestire attivamente la selezione di un nuovo colore da parte dell'utente, è necessario implementare la specifica interfaccia `ChangeListener()`

☰ Esempio di color chooser

Color Chooser



- Impostare il BorderLayout
- Aggiungere una JLabel (jIColor) a nord e impostare la proprietà text («COLOR!!!»), aumentare la dimensione del font
- Aggiungere un JColorChooser (jColorChooser) al centro

```

L      */
      public ColorChooser() {
          initComponents();
          init();
      }

      private void init() {
          jColorChooser.setColor(Color.black);
          jColorChooser.getSelectionModel().addChangeListener(new MyColorChangeListener());
      }

      protected class MyColorChangeListener implements ChangeListener {

          @Override
          public void stateChanged(ChangeEvent e) {
              ColorSelectionModel selModel = (ColorSelectionModel) e.getSource();
              jIColor.setForeground(selModel.getSelectedColor());
          }
      }
  
```

Intercettiamo la selezione di un nuovo color e cambiamo il colore della label (jIColor)

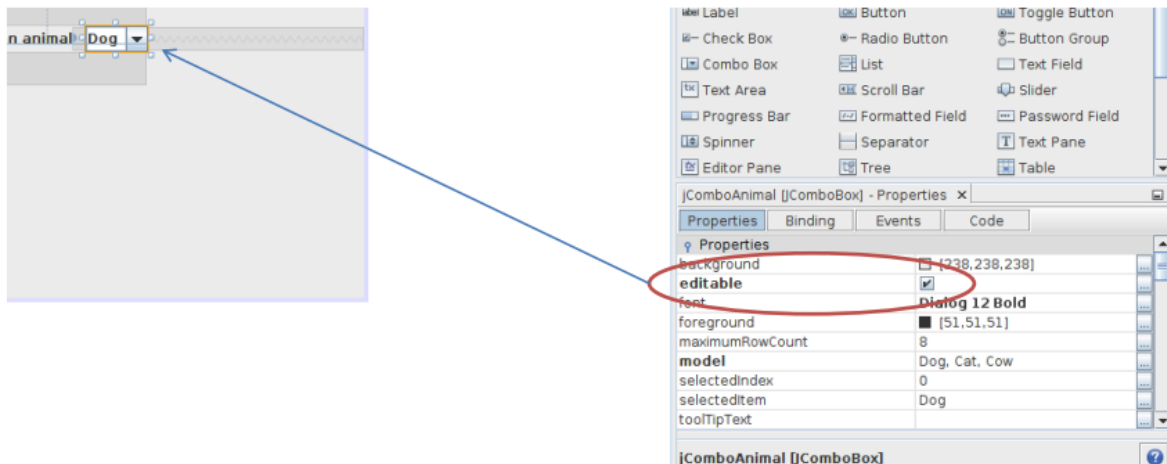
ComboBox

Il componente Combo Box consente all'utente la selezione di un valore all'interno di una serie di opzioni predefinite ed è gestito dalla classe `JComboBox`.

Questa componente può essere configurata come non editabile, vincolando la scelta alle sole voci presenti nell'elenco, oppure editabile, permettendo l'inserimento manuale di un valore da parte di un utente

Esempio di ComboBox editabile

Combo Box Editabile



Rendiamo la combo box editabile

La visualizzazione degli elementi avviene tramite un renderizzatore di default che mostra stringhe o icone, oppure richiama il metodo `toString()` nel caso di altri tipi di oggetti più complessi.

E' possibile modificare il render realizzando una classe che implementi:

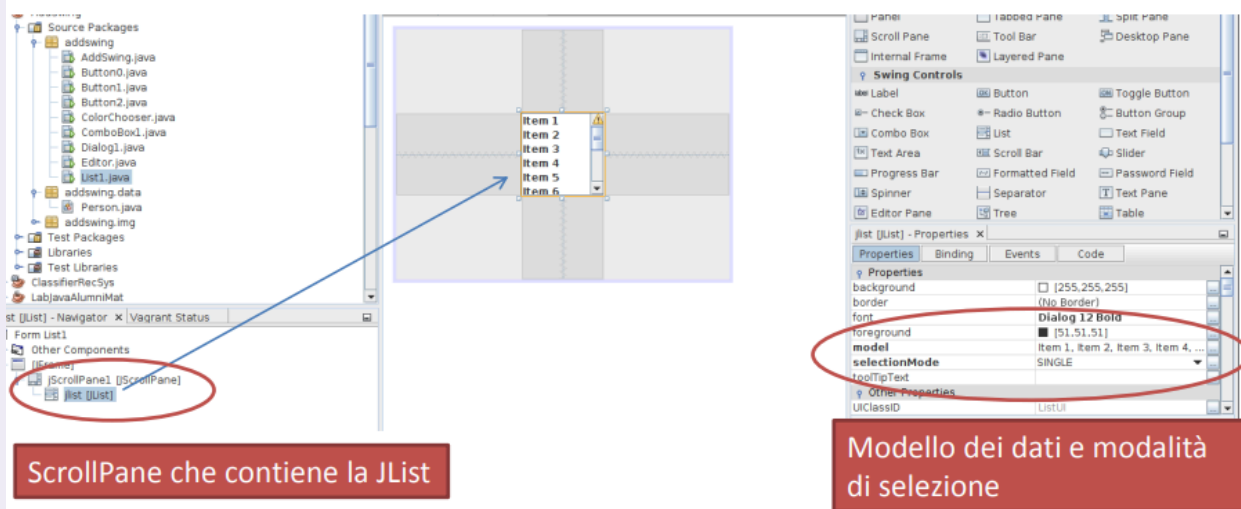
- L'interfaccia `ListCellRenderer` per le combo box non editabili,
- L'interfaccia `ComboBoxEditor` per quelle editabili

List

La lista è utile per poter visualizzare gli elementi su più righe o colonne in maniera aperta. Viene implementata dalla classe `JList` e in genere viene inserita all'interno di uno `ScrollPane` (contenitore scorrevole).

Esempio di lista

List



La lista supporta la selezione di un singolo elemento, selezioni multiple libere o selezioni basate su intervalli multipli. La manipolazione formale dei dati della lista, come l'aggiunta o la rimozione di elementi in tempo reale, viene delegata all'oggetto `DefaultListModel` associato, implementando metodi specifici come `addElement()`, `add()`, `remove()`, e `removeElement()`.

Finestre di dialogo

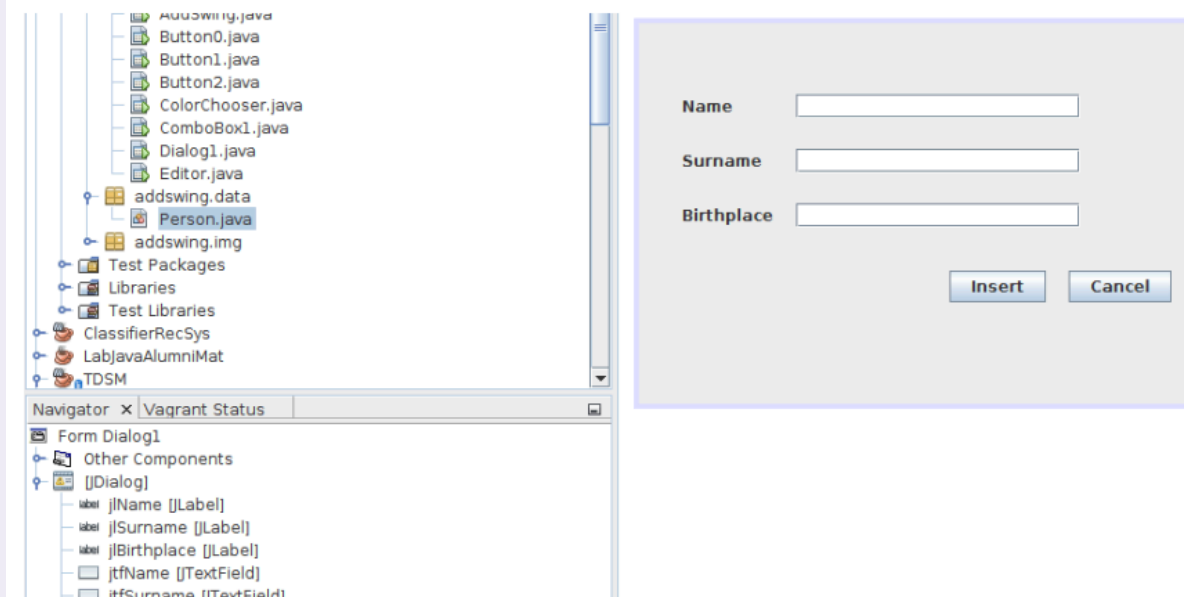
Le finestre di dialogo sono interfacce temporanee di tipo secondario, utilizzate primariamente per notificare informazioni all'utente, richiedere l'inserimento di determinati valori o far effettuare scelte rapide di esecuzione.

La classe predefinita `JOptionPane` offre un'interfaccia per invocare rapidamente finestre standardizzate capaci di visualizzare messaggi o mostrare opzioni preconfigurate (come ad esempio YES, NO e CANCEL).

È possibile creare finestre di dialogo altamente personalizzate estendendo direttamente la classe `JDialog`.

☰ Esempio di finestra di dialogo per una persona

Dialog per Person



Esempio completo su [3.2 - \(Slide Esempi\) Finestra di Dialogo Person.pdf](#)

Dal punto di vista relazionale, queste finestre dipendono sempre da un componente generatore Frame, ed esse possono inoltre essere dichiarate di tipo modale: in questo particolare stato, l'utente non può interagire con qualsiasi altra finestra dell'applicazione fino a quando il dialogo non viene risolto e chiuso.

Componenti per il testo

I componenti testuali costituiscono la base per permettere all'utente la visualizzazione e la modifica di informazioni sotto forma testuale.

Dal punto di vista gerarchico, essi derivano tutti dalla classe capostipite `JTextComponent` e si dividono convenzionalmente in tre macrocategorie principali:

- **Text Controls:** include campi semplici per l'inserimento limitato a una singola riga, come il `JTextField`, il `JPasswordField` (per offuscare i dati sensibili) e il `JFormattedTextField`. Questi componenti controllano piccole quantità di dati e generano eventi di tipo Action alla conclusione dell'inserimento testuale.
- **Plain Text Areas:** è rappresentata principalmente dal `JTextArea`, ideale per la manipolazione di testi non formattati su più righe e renderizzati utilizzando un unico font a livello globale
- **Styled Text Areas:** include il `JEditorPane` e il suo derivato avanzato `JTextPane`. Tali componenti, pur richiedendo maggiore sforzo di configurazione, supportano stili multipli di carattere, l'integrazione di componenti, di immagini e permettono di effettuare facilmente la lettura di testo formattato in HTML proveniente direttamente da un URL.

☰ Dove trovare gli esempi per questo sotto capitolo

Li si può ritrovare nelle slide prese dal professore [3.3 - \(Slide Esempi\) Esempi di componenti per il testo.pdf](#)

JTextComponent

L'architettura interna di un `JTextComponent` adotta un approccio strutturato, infatti prevede:

- Un modello di dati, detto **document**, che gestisce il contenuto
- Una vista che si occupa della visualizzazione del contenuto
- Un controller, detto **editor kit**, che legge e scrive il testo e permette le funzionalità di editing
- Un cursore che permette la navigazione nel contenuto

Document

Il modello dei dati di un `TextComponent` è il `Document` (interfaccia `Document`), che contiene il testo suddiviso in oggetti di tipo `Element`.

Questo modello di dati ha diverse funzionalità utili:

- Supporta l'editing dei documenti
- Può notificare gli eventi di modifica del testo
- Gestisce oggetti di tipo `Position` che tracciano delle porzioni di testo anche se vengono modificate
- Fornisce varie informazioni sul testo

Il `Document` notifica tutti gli oggetti `DocumentListener` registrati (collegati tramite il metodo `addDocumentListener`) a ogni occorrenza di evento di inserimento, rimozione o modifica formale dello stile.

Il listener `CaretListener` ha la funzione di ricevere tutti gli eventi correlati alle modifiche della posizione o alla selezione attuata dal cursore, e richiede di essere associato direttamente all'oggetto `JTextComponent`.

Action

Nel design dell'interfaccia, capita spesso che la medesima funzionalità (si pensi, a titolo di esempio, all'operazione di copia di un elemento) debba risultare accessibile all'utente da una pluralità di controlli diversi, come una specifica voce di menu, un'icona sulla toolbar o la pressione di una combinazione predefinita di tasti.

Al fine di non frammentare la logica di programmazione, è buona pratica di ingegneria del software procedere creando un'unica istanza di `Action` centralizzata da associare a ciascun punto di controllo.

La creazione operativa avviene estendendo la classe astratta `AbstractAction` e

sovrascrivendo, al suo interno, l'implementazione obbligatoria del metodo

`actionPerformed()` affinché contenga la corretta logica applicativa.

Fare uso di una classe `Action` permette non solo il riuso del codice, ma garantisce di poter uniformare su tutti i controlli le medesime proprietà condivise, come il testo visibile a schermo, le stringhe descrittive destinate al tooltip o le icone associate; consente peraltro di abilitare o disabilitare contemporaneamente e in modo unitario la funzionalità su tutta l'applicazione.

Lambda Expressions

Per comprendere cosa sono e come utilizzare le Lambda expressions utilizziamo un esempio legato all'utilizzo delle **classi anonime**.

Supponiamo di avere la seguente classe che memorizza delle informazioni relative ad una persona:

```
public class Person{
    public enum Gender{
        MALE, FEMALE
    }
    private String name;
    private String surname;
    private int age;
    private Gender gender;

    public Person(String name, String surname, int age, Gender gender){
        this.name=name;
        this.surnae=surname;
        this.age=age;
        this.gender=gender;
    }
    public String getName(){
        return name;
    }

    public void setGender(Gender gender){
        this.gender=gender;
    }
    public void printPerson(){
        System.out.println("Person{" + "name="+name+ ", surname="+surname+
            ", age="+age+", gender="+ gender+'}'});
    }
}
```



```

    }
}

```

Un'ulteriore modifica utile sarebbe quella di cercare delle persone della classe `Person` che rispettano alcune caratteristiche.

☰ Vogliamo cercare le persone di una certa età

```

public static void printPersonsOlderThan(List<Person> roster, int age)
{
    for (Person p: roster){
        if(p.getAge()>=age){
            p.printPerson();
        }
    }
}

```

Come si può notare, questo codice non è elegante poiché **fortemente dipendente** dalla classe `Person` e in caso di modifiche dovremmo modificare tutta la classe `Person` stessa.

Un'altra soluzione sarebbe quella di eseguire una ricerca come funzione ma anche in questo caso il metodo sarebbe legato all'attributo `age` che è memorizzato in `Person` e non è generico poiché possiamo filtrare solo tramite l'attributo età.

Possiamo definire un'interfaccia `CheckPerson` che ha un unico metodo che verifica se una istanza sia valida o meno:

```

public interface CheckPerson{
    boolean test(Person p);
}

// metodo che esegue questa interfaccia per vedere se una persona rispetta
// i criteri definiti nell'implementazione del metodo test
public static void printPerson(List<Person> roster, CheckPerson tester){
    for(Person p: roster){
        if(tester.test(p)){
            p.printPerson();
        }
    }
}

```

```

    }
}

```

Creato ciò, andiamo ad implementare l'interfaccia `CheckPerson` :

```

public class CheckPersonEligibleForSelectiveService implements
CheckPerson{
    @Override
    public boolean test(Person p){
        return p.getGender()==Person.Gender.MALE
            && p.getAge(>=18
            && p.getAge(<=25;
    }
}

```

questa nuova classe sarà utilizzata per creare una nuova istanza del suo tipo: `new CheckPersonEligibleForSelectiveService()` e passiamo l'istanza al metodo `printPerson`.

Ricerca classe anonima

Definizione di classe anonima

Una classe **anonima** viene definita tale poiché è priva di un nome essendo dichiarata e contemporaneamente istanziata.

La classe anonima può essere definita nel corpo di un metodo e sono spesso utilizzate con SWING per creare le **ActionListener**.

Si costruisce nel seguente modo:

```

//use of an anonymous class
printPerson(person, new CheckPerson(){
    @Override
    public boolean test(Person p){
        return p.getGender()==Person.Gender.MALE && p.getAge(>=18 &&
p.getAge(<=25;
    }
});

```

Interfacce funzionali

L'interfaccia `CheckPerson` usata prima è in verità un'**interfaccia funzionale**, essa differisce da un'interfaccia classica poiché:

- contiene solo un metodo astratto senza metodi statici al suo interno
- è **priva di nome** avendo solo un metodo astratto che sarà intrinsecamente implementato

Quindi un'alternativa all'uso di una classe anonima è la possibilità di usare una **lambda expression**.

Espressione lambda

Usando nuovamente l'esempio precedente vediamo come inserire l'espressione:

```
public static void printPersonWithPredicate(List<Person>
roster, Predicate<Person> tester){
    for (Person p: roster){
        if(tester.test(p)){
            p.printPerson();
        }
    }
}
```

La voce `Predicate<T>` è un **interfaccia funzionale** definita in `java.util.function` ed esegue la stessa funzione che eseguiva precedentemente `CheckPerson`, ma essendo **generica** e avendo solo il suo stesso metodo: `boolean test(T t)` non è necessario specificare il suo nome e può essere utilizzata in un'espressione lambda nel seguente modo:

```
printPersons(persons,
    (Person p) -> p.getGender() == Person.Gender.MALE // Predicate function
    && p.getAge() >= 18
    && p.getAge() <= 25
);
```

lambda expression

Sintassi di Lambda

L'espressione lambda è composta:

- da una **lista di parametri formali** separati da virgole e racchiusi tra parentesi tonde.
- il token `→`
- un **corpo** che contiene una singola espressione o un blocco di istruzioni. Il blocco di istruzioni è racchiuso nelle parentesi graffe {}, se il blocco di istruzioni contiene un'invocazione ad un metodo che non restituisce nessun valore (void) si possono omettere le parentesi.

⚠ Nota bene

È possibile omettere il tipo di dati dei parametri in un'espressione lambda. Inoltre, è possibile omettere le parentesi se esiste un solo parametro

☰ Visualizziamo il seguente esempio con più parametri

```
public class Calculator{
    interface IntegerMath{
        int operation(int a,int b);
    }

    public int operateBinary(int a,int b,IntegerMath op){
        return op.operation(a,b);
    }

    public static void main(Strng...args){
        Calculator myApp=new Calculator();
        IntegerMath addition=(a,b) -> a+b; //parametro formale con
        lambda
        IntegerMath subtraction=(a,b) -> a-b; //parametro formale
        con lambda
        System.out.println("40+2=
        "+myApp.operateBinary(40,2,addition));
        System.out.println("20-10=
        "+myApp.operateBinary(20,10,subtraction));
    }
}
```

Perché usare le lambda expressions?

Le lambda expressions sono un ottimo esempio di **programmazione funzionale**:

- Privo di **side-effect**
- Flusso di esecuzione a valutazione di funzioni

Programmazione funzionale:

- La programmazione funzionale si concentra sulla **definizione di funzioni**
contrariamente, i paradigmi procedurali e imperativi prediligono la specifica di una

sequenza di comandi da eseguire e i valori vengono calcolati cambiando lo stato del programma attraverso l'operazione di assegnazione.

- La programmazione funzionale basa le sue radici nel **lambda calcolo**, ossia un **calcolo basato sulle funzioni**, composto da un linguaggio formale utilizzato per esprimere le funzioni e un sistema di riscrittura per stabilire come i termini possano essere ridotti e semplificati.

Consumer

Riprendendo il metodo `printPersonWithPredicate`. Il metodo è ancora più **generalizzabile**, attuando una generalizzazione sull'**operazione** da applicare alle istanze per cui `test` da `true`.

Per farlo utilizziamo `Consumer<T>`. Questa interfaccia è definita in `java.util.function`, il suo funzionamento è identico a `printPerson`. L'unico metodo di `Consumer` è `void accept(T t)`.

Se volessimo effettuare un'operazione su un'istanza prima di *consumarla* necessitiamo dell'interfaccia **function**.

Function

Anche quest'interfaccia è definita in `java.util.function` ed è dichiarata nel seguente modo: `Function<T, R>`. Questa interfaccia esegue un'operazione sull'istanza *T* restituendo un'istanza *R*.

Anche `Function` ha un unico metodo, il quale è `R apply(T t)`.

```
//Function<T, R> è una funzione predefinita che agisce come R apply(T t)
//Apply permette di eseguire un'azione in T restituendo R
public static void processPersonWithFunction(List<Person>
roster, Predicate<Person> tester, Function<Person, String> mapper,
Consumer<String> block) {
    //mapper trasforma un oggetto in un altro
    for (Person p : roster) {
        if (tester.test(p)) {
            String data = mapper.apply(p);
            block.accept(data);
        }
    }
}

processPersonWithFunction(persons,
    p -> p.getGender() == Person.Gender.MALE && p.getAge() >= 18 && p.getAge()
```

```

    <=25,
    p->p.getSurname(), //Function
    surname->System.out.println(surname) //Consumer
);

```

Visibilità delle variabili

Un'espressione lambda non aggiunge un nuovo livello di **scope**, infatti sono definite **lexically scoped** e si comportano di fatto come le classi anonime, quindi accedendo alle variabili definite nello scope locale nel quale è scritta l'espressione lambda.

Generics nelle Lambda Expression

Il metodo `checkPersonWithFunction` è possibile renderlo ancora più **generico** così da poterlo applicare a qualsiasi tipo di classe, senza essere così vincolato da `Person` e contemporaneamente è generico anche per il tipo restituito dall'implementazione di `Function`

```

public static <X,Y> void processElements(Iterable<X> source, Predicate<X>
tester,      Function<X,Y> mapper, Consumer<Y> block){
    for(X p:source){
        if(tester.test(p)){
            Y data=mapper.apply(p);
            block.accept(data);
        }
    }
}

```

Qui entrano in gioco nuovi simboli e attori:

- X è il **generics** che fa riferimento agli oggetti che voglio processare
- `Iterable<X>` mi permette di iterare l'operazione che voglio eseguire su tutti gli oggetti di tipo X
- Y è il **generics** relativo al risultato della funzione mapper (ovvero della `Function`)

Segue l'esempio di due azioni da eseguire su `processElements`

Sono due funzioni diverse che utilizzano lo stesso metodo generalizzato precedentemente per cercare il *cognome* e l'*età*.

```

processElements(persons,
    p->p.getGender()==Person.Gender.MALE && p.getAge()>=18 &&
    p.getAge()<=25,

```

```

    p->p.getSurname(), //Function
    surname->System.out.println(surname) //Consumer
);

processElements(persons,
    p->p.getGender() == Person.Gender.MALE && p.getAge() >= 18 &&
    p.getAge() <= 25,
    p->p.getAge(), //Function
    age->System.out.println(age) //Consumer
);

```

Nel dettaglio il metodo `processElements` lavora in questo modo:

- **Ottiene** una sorgente di oggetti da un iteratore. Nell'esempio gli oggetti sono di tipo `Person` e la sorgente è una `List` che implementa l'interfaccia `Iterable`
- **Filtra** gli oggetti in base all'oggetto `Predicate`. Nel nostro caso definito dall'espressione lambda che controlla il **genere e l'età**
- **Mappa** tutti gli oggetti filtrati su un altro valore definito da `Function`. In questo caso un'altra espressione lambda che restituisce il **cognome**
- **Esegue** un'azione sull'oggetto mappato definita dall'oggetto `Consumer`. In questo caso stampa una stringa che è il cognome dell'oggetto restituito da `Function`

Quello che esegue il metodo `ProcessElements` è ottenibile attraverso l'uso degli **stream** e delle **operazioni aggregate**.

```

//processElements viene rimpiazzato con un pipeline applicata su uno stream
person
    .stream()
    .filter(p->p.getGender() == Person.Gender.MALE
    && p.getAge() >= 18
    && p.getAge() <= 25)
    .map(p->p.getSurname())
    .forEach(surname->System.out.println(surname));

```

Le seguenti operazioni: `.map`, `.filter` e `forEach` sono **operazioni aggregate** che processano gli elementi a partire da uno **stream**.

Pipeline e stream

Uno **stream** è una **sequenza di elementi** che differisce dalla **collection** poiché non è una struttura dati che può contenere elementi.

Lo stream preleva i valori da una **sorgente** attraverso una **pipeline** (una sorgente può essere una collection stessa).

Pipeline

In Java una **Pipeline** è una **sequenza di operazioni** (come quelle viste precedentemente), dove generalmente i **parametri** di queste operazioni sono **lambda expression** e quindi facilmente personalizzabili.

Andando ad analizzare nel dettaglio lo stream dell'esempio precedente si hanno due operazioni:

- `filter` che filtra gli oggetti
- `forEach` che esegue le operazioni su tutti gli elementi dello stream

In questo caso la pipeline di questo stream se la volessimo analizzare senza la sua aggregazione sarebbe:

```
for(Person p:roster){
    if(p.getGender()==Person.Sex.MALE){
        System.out.println(p.getName());
    }
}
```

Concettualmente possiamo suddividere la composizione di una pipeline in:

- **Sorgente**, che potrebbe essere una collection, un array, un canale di I/O, ecc...
- Zero o più **operazioni intermedie**, un esempio di operazione intermedia è `filter`. Ogni operazione intermedia genera un nuovo `stream`
- **Operazione terminale**. L'opzione terminale è una conclusione della pipeline che produce un **risultato finale** che non è più uno stream.

In allegato il link della Javadoc con la lista delle operazioni terminali:

<https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html>

Funzioni generatrici

`Stream` ha un metodo `generate` che permette di generare uno stream all'infinito di valori la cui generazione è dipesa dal parametro di tipo `Supplier`: `generate(Supplier<T> s)`

L'interfaccia di `Supplier` possiede solamente un metodo `get` che restituisce un valore `T`.

L'altro metodo di `Stream` che genera valori è `iterate` che genera valori infiniti a partire da un seed al quale viene applicato iterativamente una funzione `f`: `seed, f(seed), f(f(seed), ecc...`

≡ Esempio di funzione generatrice

Crea uno stream infinito di numeri random nell'intervallo $[0, 1)$, filtra i primi 100 maggiori o uguali a 0.5 e li stampa.

```
DoubleStream.generate(()->Math.random())
    .filter(p->p>=0.5)
    .limit(100)
    .forEach(p->System.out.println(p));
```

In questo caso troviamo la generazione con `generate(()->Math.random())`. La funzione `Supplier` è definita attraverso un'espressione **lambda** senza parametri, `()`, che esegue un'istruzione che genera un double, in questo caso il metodo statico `random` di `Math`.

≡ Esempio di funzione generatrice

Genera 100 numeri random nell'intervallo $[0, 1)$, filtra quelli maggiori a 0.5 e conta quanti sono. Il risultato finale viene stampato.

```
System.out.println(DoubleStream.generate(()->Math.random())
    .limit(100)
    .filter(p->p>0.5)
    .count());
```

Reduction

Le operazione come `avg` e `count` (vista nell'esempio precedente) sono dette **riduzioni**. Queste restituiscono un unico valore combinando gli oggetti presenti nello stream ma possono esistere anche alcuni tipi di riduzioni che restituiscono una collection e non un singolo valore.

≡ Esempi di operazioni di riduzione che restituiscono un singolo valore:

`average`, `sum`, `min`, `max` e `count`

Oltre alle operazioni di riduzione predefinite è possibile definirne di **nuove utilizzando i metodi di Stream**: `reduce` e `collect`.

Stream reduce

Analizziamo un codice in cui si vuole calcolare la somma di tutte l'età presenti utilizzando la riduzione `sum`:

```
Integer totalAge=persons
    .stream()
    .mapToInt(p->p.getAge())
    .sum();
System.out.println(totalAge);
```

Questa funzione può essere esattamente modificata con `reduce` al posto di `sum`:

```
totalAge=persons
    .stream()
    .mapToInt(p->p.getAge())
    .reduce(0, (a, b)->a+b);
System.out.println(totalAge);
```

Analizziamo nel dettaglio cosa è successo. In primis si nota chiaramente che `reduce` riceve in input due parametri:

- **identify**: questo valore (nel nostro caso 0) è il valore da restituire in caso lo stream fosse vuoto e da dove inizierà la sua esecuzione il metodo `reduce` all'interno dello Stream.
- **accumulator**: la funzione accumulatore accetta **due parametri dello stesso tipo e restituisce un risultato**. In questo esempio, la funzione accumulatore è un'espressione lambda che aggiunge due valori Integer(`a` , `b`) e restituisce un valore Integer ($a + b$). I due parametri della funzione accumulatore sono: il **risultato parziale della riduzione** (in questo esempio, la somma di tutti gli interi elaborati finora) e l'**elemento successivo dello stream** (in questo esempio, un numero intero, l'età).

Stream collect

`Collect` risolve il problema generato da `reduce`. Se ricordiamo infatti, la programmazione funzionale è priva di **side-effect**, ma con l'esempio precedente abbiamo sempre un effetto secondario, ossia la restituzione di un nuovo valore.

Nel caso precedente il nuovo valore è di tipo Integer quindi non abbiamo grossi problemi di performance, ma se la funzione reduce lavorasse con **oggetti più complessi**, ad esempio Collection, ogni volta che viene chiamata creerebbe una nuova Collection.

In questi casi è opportuno utilizzare il metodo `collect` che effettua un **update dell'oggetto** che si sta ottenendo dalla riduzione dello stream.

⚠ Prestare attenzione all'utilizzo

Da quello che si è detto si evince che `collect` sovrascrive il valore preesistente estrapolato dallo stream, contrariamente da `reduce` che in casi lievi lavora creando nuovi dati.

Il metodo `collect` accetta **tre parametri**:

1. **supplier**: è il costruttore dell'oggetto che verrà restituito dal metodo `collect` e che si presuppone venga modificato durante l'operazione di riduzione
2. **accumulator**: questa funzione serve ad inglobare il valore attuale dello stream nell'oggetto che verrà restituito
3. **combiner**: questa funzione combina due contenitori di risultati e unisce il loro contenuto

☰ Costruiamo qualcosa di complesso per visualizzare un ottimo esempio.

Supponiamo di voler implementare l'operatore terminale che calcola la media attraverso il metodo `collect`. Abbiamo bisogno di definire una classe `Average` che manterrà la somma corrente e il numero totale degli elementi esaminati nello stream. Questa classe verrà poi utilizzata nel metodo `collect`

```
public class Average implements IntConsumer{
    private int sum=0;
    private int count=0;

    public double average(){
        return count>0?((double)sum) / count:0
    }

    @Override
    public void accept(int value){
        sum+=value;
        count++;
    }

    public void combine(Average other){
        sum+=other.sum;
        count+=other.count;
    }
}
```

```
}
}
```

La classe `Averager` implementa l'interfaccia `IntConsumer` che prevede il solo metodo `accept` che verrà utilizzato come **accumulator**. Il **costruttore** sarà il metodo utilizzato come **supplier**, invece il metodo `combine` come **combiner**.

Il costruttore ci restituirà il risultato finale.

La sua implementazione in un codice è la seguente:

```
Averager avgAgeC=persons
    .stream()
    .map(p->p.getAge())
    .collect(Averager::new, Averager::accept, Averager::combine);
System.out.println(avgAgeC.average());
```

Come possiamo notare c'è un nuovo **operatore (::)** in questa implementazione, chiamato **method reference operator**. Questo nuovo operatore si comporta come alle espressioni lambda che invocano direttamente il metodo, ma è più immediato poiché basato solo sul **nome del metodo**.

In breve, con questo operatore stiamo andando ad utilizzare i metodi di una classe facendo riferimento direttamente con il nome di quest'ultima.

Se volessimo estendere quella sezione avremmo dovuto scrivere:

```
.collect(()->new Averager(), (a,b)->a.accept(b), (a,b)->a.combine(b));
```

Group By

Il collector `groupingBy` permette di raggruppare gli oggetti all'interno di uno stream in base a una specifica **funzione di classificazione**. L'operazione restituisce tipicamente una `Map`.

≡ **Esempio base di Group By** Nell'esempio seguente filtriamo tutte le persone con età maggiore a 17 e le raggruppiamo in base al genere.

```
Map<Person.Gender, List<Person>> collect = persons
    .stream()
    .filter(p -> p.getAge() > 17)
    .collect(Collectors.groupingBy(Person::getGender));
```

Esiste anche un'implementazione di `groupBy` che accetta **due parametri**: il primo è la funzione di classificazione, mentre il secondo è chiamato **downstream collector**, ovvero un ulteriore collector che viene applicato al risultato del raggruppamento.

≡ Esempio di Group by con downstream collector (`mapping`)

Il Collector `mapping` utilizzato come downstream produce una `List` dove il tipo degli elementi dipende dalla funzione passata come primo argomento.

In questo caso raggruppiamo per genere e inseriamo nella lista **solo il nome**.

```
Map<Person.Gender, List<String>> namesByGender = persons
    .stream()
    .collect(
        Collectors.groupingBy(
            Person::getGender,
            Collectors.mapping(
                Person::getName,
                Collectors.toList()
            )
        )
    );
```

≡ Esempio di Group by con downstream collector (`reducing`)

In questo caso il downstream collector è un'operazione di **riduzione**.

Nell'esempio si effettua la somma delle età raggruppandole per genere.

```
Map<Person.Gender, Integer> totalAgeByGender = persons
    .stream()
    .collect(
        Collectors.groupingBy(
            Person::getGender,
            Collectors.reducing(
                0,
                Person::getAge,
                Integer::sum
            )
        )
    );
```

```
    )
);
```

Esecuzione parallela

Le operazioni sugli stream possono essere eseguite in parallelo (sfruttando il multithreading e l'esecuzione concorrente) semplicemente sostituendo il metodo `.stream()` con `.parallelStream()`.

≡ Esempio di calcolo della media in esecuzione parallela

```
double average = persons
    .parallelStream()
    .filter(p -> p.getGender() == Person.Gender.MALE)
    .mapToInt(Person::getAge)
    .average()
    .getAsDouble();
```

≡ Esempio di raggruppamento concorrente

```
ConcurrentMap<Person.Gender, List<Person>> byGender = persons
    .parallelStream()
    .collect(
        Collectors.groupingByConcurrent(Person::getGender)
    );
```

Per i raggruppamenti paralleli si utilizza il collector specifico `groupingByConcurrent`, che restituisce una `ConcurrentMap`.

Come usarle e quando?

Quando c'è bisogno di passare l'implementazione di un'interfaccia a singolo metodo, la soluzione non sono le classi anonime ma le **lambda expression** stesse.